

第8章 函 数

Part II

7.9变量的存储方式和生存期

7.9.1 动态存储方式与静态存储方式

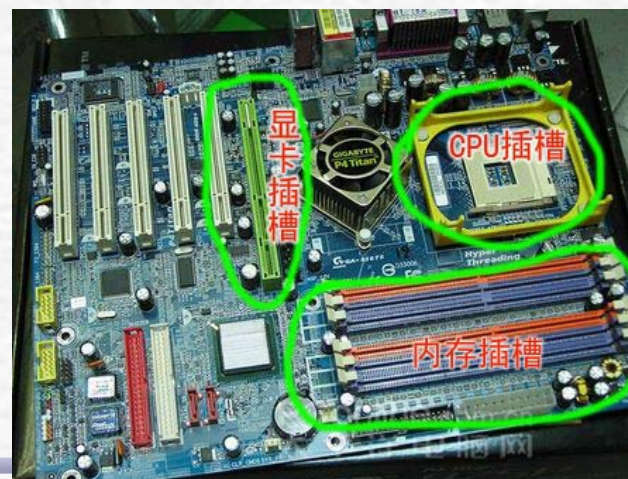
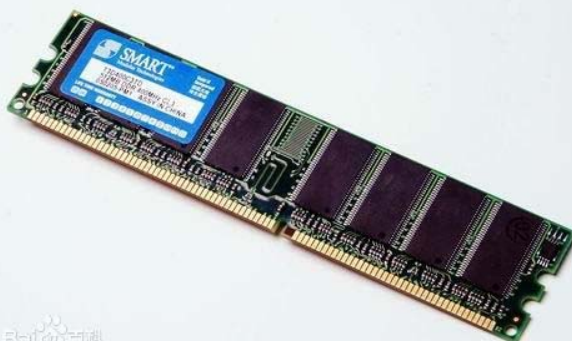
7.9.2 局部变量的存储类别

7.9.3 全局变量的存储类别

7.9.4 存储类别小结

内 存 储 器

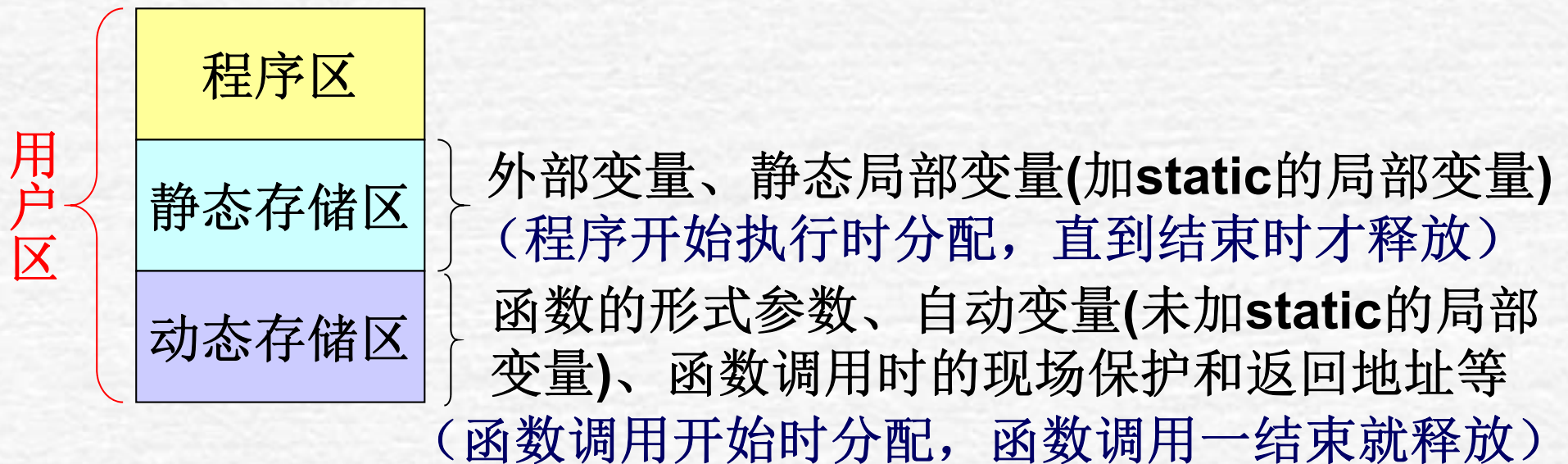
- 内存储器是硬盘等外部设备与CPU进行沟通的桥梁，它的性能对计算机的影响非常大。
 - 计算机中所有程序的运行都是在内存储器中进行的。
 - 还用于暂时存放CPU中的运算数据，以及与硬盘等外部设备交换的数据。
- 内存储器包括寄存器、高速缓冲存储器（Cache）和主存储器（俗称内存）。
 - 寄存器在CPU芯片的内部，高速缓冲存储器也制作在CPU芯片内，而主存储器由插在主板内存插槽中的若干内存条组成。
 - 内存常采用半导体存储器，特点：
 1. 可以读出/写入，读出时并不损坏原来存储的内容，只有写入时才修改原来所存储的内容；
 2. 只能用于暂时存放信息，一旦断电，存储内容立即消失



7.9.1 变量的存储类别

从变量值存在的时间(即生存期)观察, 可分为:

- 静态存储方式 —— 指在程序运行期间分配固定的存储空间的方式。
- 动态存储方式 —— 在程序运行期间根据需要进行动态的分配存储空间的方式。
- 内存中供用户使用的存储空间的情况:



每一个变量和函数都有两个属性：**数据类型**和数据的**存储类别**

- **数据类型**，如整型、浮点型等
- **存储类别**指的是数据在内存中存储的方式(如静态存储和动态存储)
 - 存储类别包括：
自动的**auto**、静态的**static**、
寄存器的**register**、外部的**extern**
 - 根据变量的存储类别，可以知道变量的作用域和生存期



7.9.2 变量的存储类别

auto变量 —— 自动变量

- 函数中的局部变量，即：
 - 函数中的形式参数
 - 在函数中定义的声明为**auto**的或未专门声明为其它存储类型的变量

- 复合语句中定义的变量

[存储区域] 动态存储区

[生存期] 在调用该函数/复合语句时分配存储空间，调用一结束就自动释放这些存储空间。

[自动变量的声明] 在函数体/复合语句中：

[auto] 数据类型 变量名

- 由于程序中大多数变量都是自动变量，因此**auto**可以省略，不写即隐含地确定为“自动存储类别”。

7.9.2 局部变量的存储类别

```
int f(int a)
{
    auto int b,c=3;
}
```

可以省略

7.9.2 变量的存储类别

8

用**static**声明局部变量 —— 静态局部变量

[存储区域] 静态存储区

[生存期] 在程序开始执行时就分配存储空间，**函数调用结束后并不释放，而是仍保留原值**，并将其作为下次调用该函数时该变量的初始值，直到程序执行完毕才释放。

[静态局部变量的声明] 在函数体/复合语句中：

static 数据类型 变量名

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

调用三次

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

每调用一次，开辟新a和b，但c不是

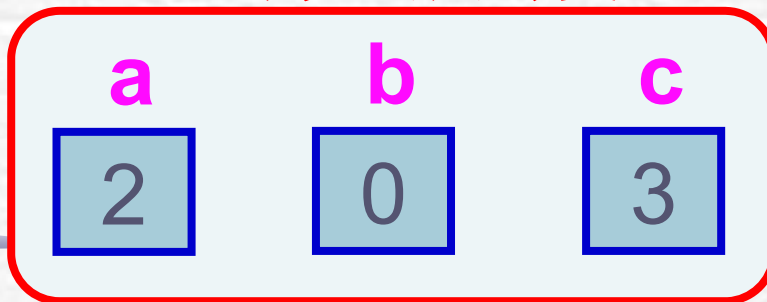
例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```



```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



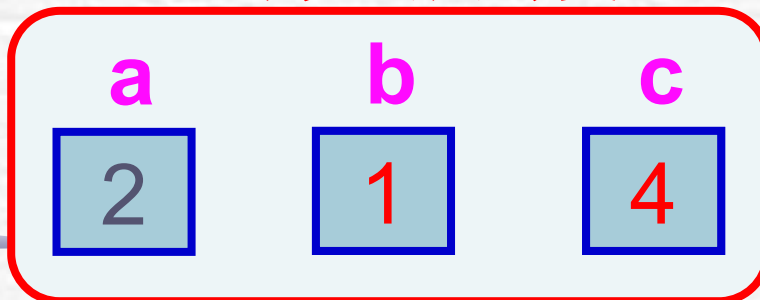
第一次调用开始

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



第一次调用期间

例7.16 考察静态局部变量的值。

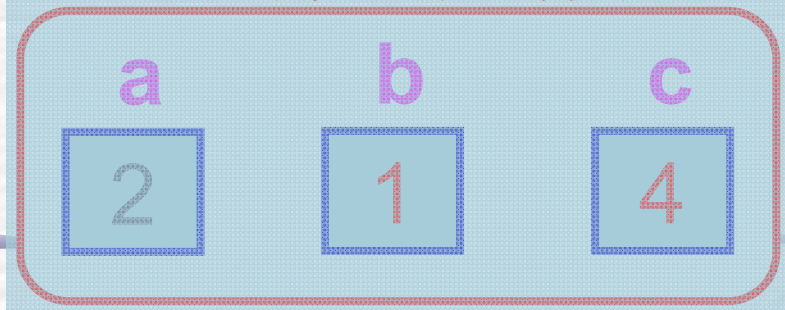
```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

7



```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



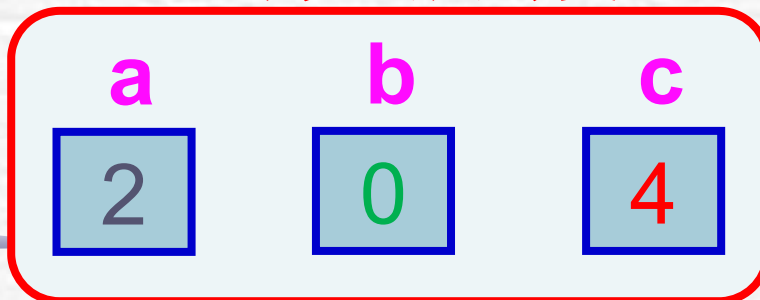
第一次调用结束

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



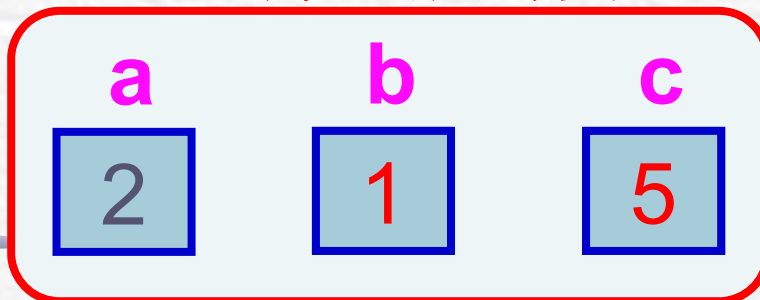
第二次调用开始

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



第二次调用期间

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

8

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量

a
2

b
1

c
5

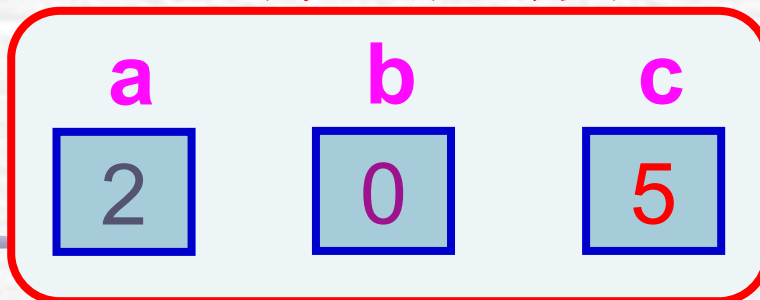
第二次调用结束

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



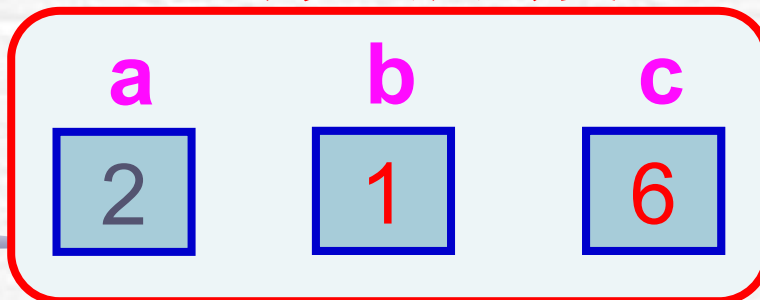
第三次调用开始

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量



第三次调用期间

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

9

函数f的局部变量

a
2

b
1

c
6

第三次调用结束

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
  return(a+b+c);
}
```

函数f的局部变量

c

6

整个程序结束

```
7
8
9
```

例7.16 考察静态局部变量

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

在函数调用时赋初值

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
}
```

在程序一开始时
赋初值

例7.16 考察静态局部变量

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

若不赋初值，不确定

```
int f(int a)
{ auto int b;
  static c;
  b=b+1;
  c=c+1;
}
```

若不赋初值，是0

例7.16 考察静态局部变量的值。

```
#include <stdio.h>
int main()
{ int f(int);
  int a=2,i;
  for(i=0;i<3;i++)
    printf("%d\n",f(a));
  return 0;
}
```

```
int f(int a)
{ auto int b=0;
  static c=3;
  b=b+1;
  c=c+1;
}
```

仅在本函数内有效

7.9.2 变量的存储类别

23

- 对静态局部变量是在程序一开始执行时就赋初值的，且只赋初值一次。以后每次调用函数时不再重新赋初值而只是保留上次函数调用结束时的值。
- 在定义局部变量时如果不赋初值，对于静态局部变量将自动被赋0，而自动变量则取一个不确定的值。
- 静态局部变量在其它函数中是不可见的。
- 主要应用：保留上次计算结果，减少重复计算
- 缺点：占用内存、降低程序可读性

例7.17 输出1到5的阶乘值。

- 解题思路：可以编一个函数用来进行连乘，如第1次调用时进行1乘1，第2次调用时再乘以2，第3次调用时再乘以3，依此规律进行下去。

```
#include <stdio.h>
```

```
int main()
```

```
{ int fac(int n);
```

```
    int i;
```

```
    for(i=1;i<=5;i++)
```

```
        printf("%d!=%d\n",i,fac(i));
```

```
    return 0;
```

```
}
```

```
int fac(int n)
```

```
{ static int f=1;
```

```
    f=f*n;
```

```
    return(f);
```

```
}
```

若非必要，不要使用静态局部变量

这是一个反例！

```
1!=1
```

```
2!=2
```

```
3!=6
```

```
4!=24
```

```
5!=120
```

7.9.2 变量的存储类别

register变量——寄存器变量

一般，变量的值是放在内存中的，这些变量值的存取需要在内存与**CPU**间交换数据。如果一些变量使用频繁，则为存取变量的值将花费不少时间。

为了提高执行效率，**C**语言允许将局部变量的值放在**CPU**中的寄存器中。这种变量叫做“寄存器变量”，用关键字**register**作声明。

- 只有局部自动变量和形式参数可以作为寄存器变量，其他（如全局变量、局部静态变量）不行。
- 寄存器变量对寄存器的使用也是动态的。
- 一个计算机系统中的寄存器数目是有限的，不能定义任意多个寄存器变量。
- 现有的优化编译系统能够识别使用频繁的变量，并将其自动放在寄存器中，因而无需人为指定。

7.9.3 全局变量的存储类别

- 全局变量都是存放在静态存储区中的。因此它们的生存期是固定的，存在于程序的整个运行过程
- 一般来说，**外部变量**是在函数的外部定义的全局变量，它的作用域是**从变量的定义处开始**，到本程序**文件的末尾**。在此作用域内，全局变量可以为程序中各个函数所引用。

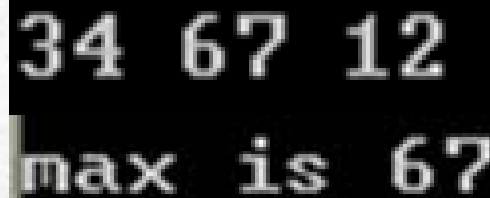
1. 在一个文件内扩展外部变量的作用域

- 外部变量有效的作用范围只限于定义处到本文件结束。
- 如果在定义点之前的函数想引用某外部变量，在引用之前用关键字**extern**对该变量作“外部变量声明”，则可以从“声明”处起，合法地使用该外部变量
 - 用**extern**声明外部变量时，类型名可以省略。

例7.18 调用函数，求3个整数中的大者。

✎ 解题思路：用**extern**声明外部变量，扩展外部变量在程序文件中的作用域。


```
#include <stdio.h>
int main()
{ int max( );
  extern int A,B,C;
  scanf("%d %d %d",&A,&B,&C);
  printf("max is %d\n",max());
  return 0;
}
int A ,B ,C;
int max( )
{ int m;
  m=A>B?A:B;
  if (C>m) m=C;
  return(m);
}
```



```
34 67 12
max is 67
```

2. 将外部变量的作用域扩展到其他文件

- 如果一个程序包含两个文件，在两个文件中都要用到同一个外部变量Num，不能分别在两个文件中各自定义一个外部变量Num
- 应在任一个文件中定义外部变量Num，而在另一文件中用extern对Num作“外部变量声明”
- 在编译和连接时，系统会由此知道Num有“外部链接”，可以从别处找到已定义的外部变量Num，并将在另一文件中定义的外部变量num的作用域扩展到本文件

例7.19 给定**b**的值，输入**a**和**m**，求 **$a*b$** 和 **a^m** 的值。

解题思路：

- 分别编写两个文件模块，其中文件**file1**包含主函数，另一个文件**file2**包含求 **a^m** 的函数。
- 在**file1**文件中定义外部变量**A**，在**file2**中用**extern**声明外部变量**A**，把**A**的作用域扩展到**file2**文件。

文件**file1.c**:

```
#include <stdio.h>
```

```
int A;
```

```
int main()
```

```
{ int power(int);
```

```
    int b=3,c,d,m; scanf("%d,%d",&A,&m);
```

```
    c=A*b;
```

```
    printf("%d*%d=%d\n",A,b,c);
```

```
    d=power(m);
```

```
    printf("%d**%d=%d\n",A,m,d);
```

```
    return 0;
```

```
}
```

编译和运行包括多个文件的程序。

文件**file2.c**:

extern A;

int power(int n)

{ int i,y=1;

for(i=1;i<=n;i++)

y*=A;

return(y);

}

13.3

13*3=39

13**3=2197

如何运行一个多文件的程序

- 用**VC**等集成环境
 - 建立项目文件 (*.prj)
- 用命令行方式
 - DOS**下的**tlink**命令
- 用**#include**预处理命令
 - 实质上，是将多个文件拼合成一个文件

- 这样使用全局变量时应当十分慎重，因为若在一个文件的函数中改变了该全局变量的值，将会影响其他文件中相关函数的执行结果。

系统如何区分**extern**的这两种用法？

1. 当编译遇到**extern**时，先在本文件中找外部变量的定义，如果找到，就在本文件中扩展作用域。
2. 如果找不到，就在连接时到其它文件中找外部变量的定义，如果找到，就将作用域扩展到本文件。
3. 如果找不到，按出错处理。

3.将外部变量的作用域限制在本文件中

- 有时在程序设计中希望某些外部变量只限于被本文件引用。这时可以在定义外部变

只能用于本文件

该文件仍然不能用

```
file1.c  
static int A;  
int main ()  
{  
    .....  
}
```

```
file2.c  
extern A;  
void fun (int n)  
{  
    .....  
    A=A*n;  
    .....  
}
```

- 这种加上**static**声明、只能用于本文件的外部变量称为静态外部变量。
- 优点：有利于模块化，避免被其它文件误用。
- 说明：
 - 不要误认为对外部变量加**static**声明后才采取静态存储方式，而不加**static**的是采取动态存储
 - 声明局部变量的存储类型和声明全局变量的存储类型的含义是不同的
 - 对于**局部变量**来说，声明存储类型的作用是指定变量存储的区域以及由此产生的生存期的问题，而对于**全局变量**来说，声明存储类型的作用是变量作用域的扩展问题

用**static** 声明一个变量的作用是：

(1) 对**局部变量**用**static**声明，把它分配在静态存储区，该变量在整个程序执行期间不释放，其所分配的空间始终存在。

(2) 对**全局变量**用**static**声明，则该变量的作用域只限于本文件模块(即被声明的文件中)。

【注意】用**auto**、**register**、**static**声明变量时，是在定义变量的基础上加上这些关键字，而不能单独使用。

下面用法不对：

```
int a;
```

```
static a;
```

//编译时会被认为“重新定义”。

7.9.4 存储类别小结

- 对一个数
- 数据类型

例如:

```
static int a;
```

```
auto char c;
```

```
register int d;
```

- 可以用extern声明已定义的外部变量

例如: `extern b;`

静态局部整型变量或
静态外部整型变量

自动变量, 在

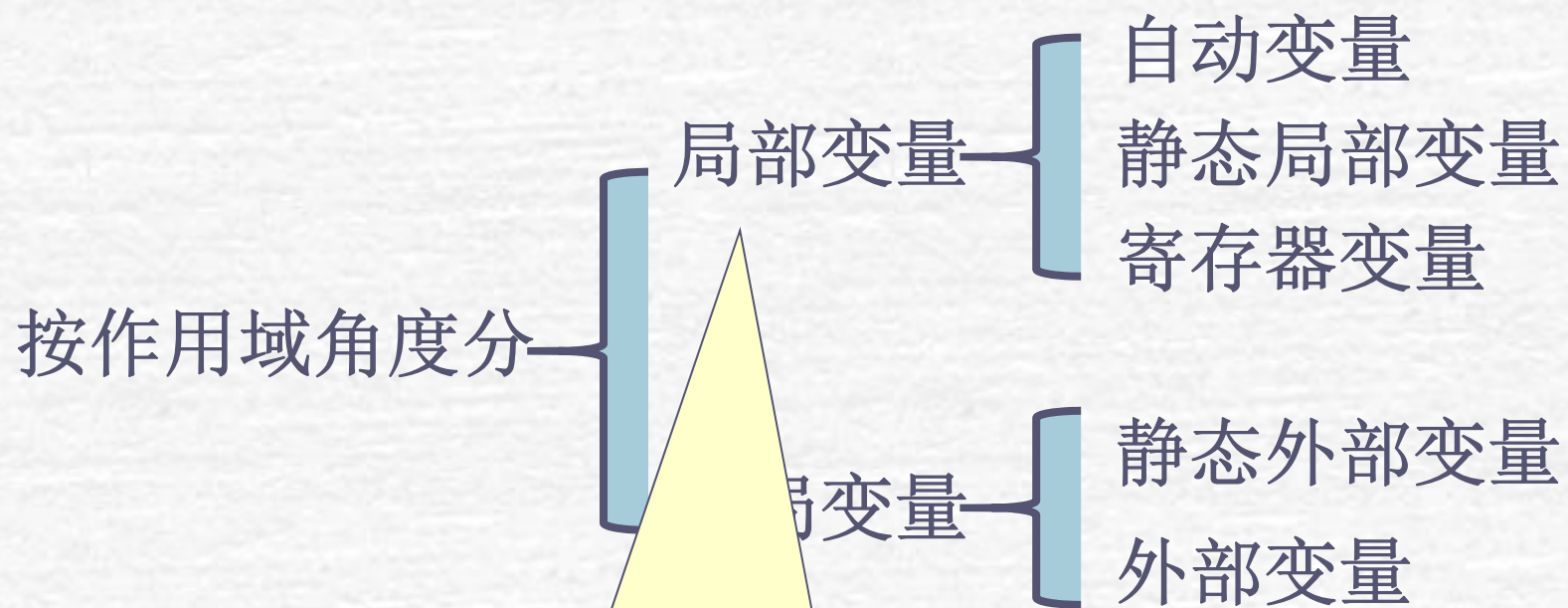
寄存器变量,
在函数内定义

将已定义的外部变量
b的作用域扩展至此

两种属性:

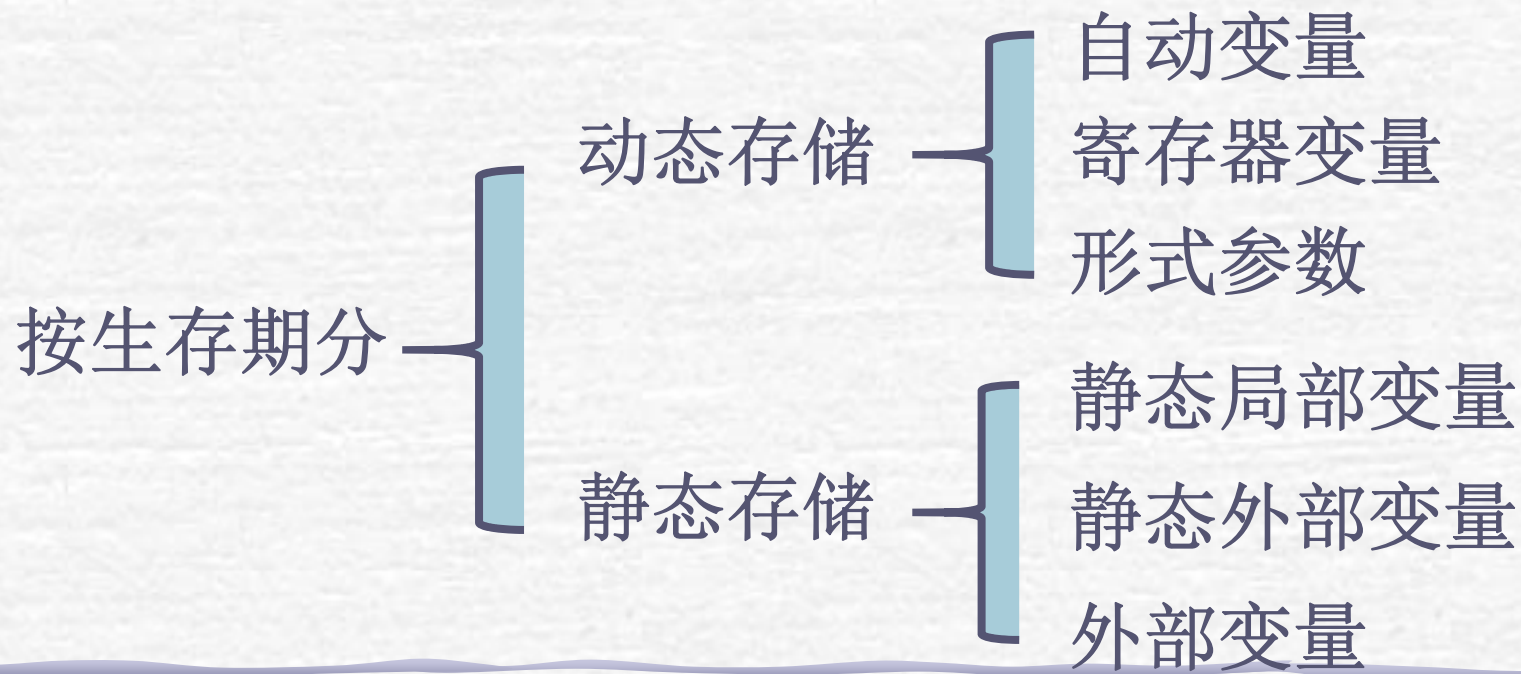
两个关键字

(1) 从作用域角度分，有局部变量和全局变量。它们采用的存储类别如下：

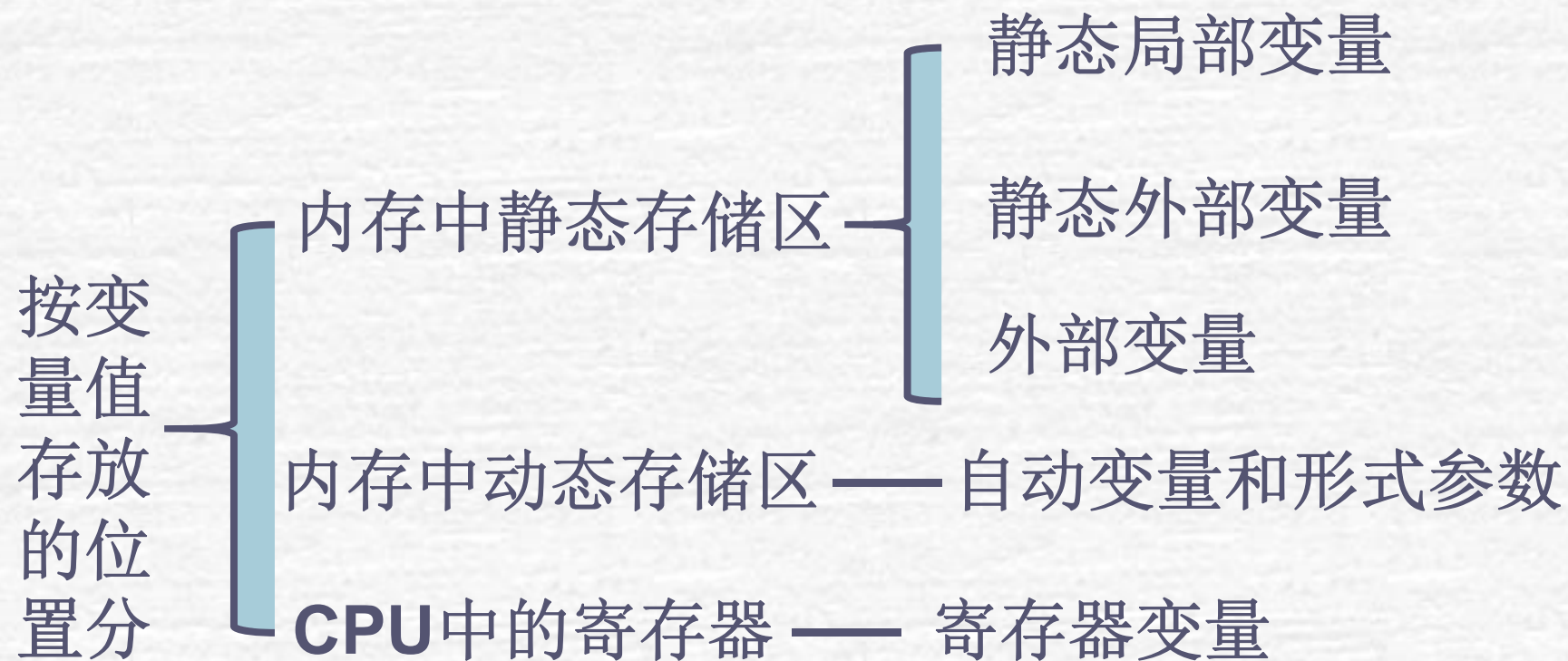


形式参数可以定义为自动变量或寄存器变量

(2) 从变量存在的时间区分,有动态存储和静态存储两种类型。**静态存储**是程序整个运行时间都存在,而**动态存储**则是在调用函数时临时分配单元



(3) 从变量值存放的位置来区分,可分为:



(4) 关于作用域和生存期的概念

对一个变量的属性可以从两个方面分析：

- 作用域：如果一个变量在某个文件或函数范围内是有效的，就称该范围为该变量的**作用域**
- 生存期：如果一个变量值在某一时刻是存在的，则认为这一时刻属于该变量的**生存期**

作用域是从**空间**的角度，生存期是从**时间**的角度

二者有联系但不是同一回事

文件file1.c

```
int a;  
int main( )  
{ ...f2( );...f1( );... }  
void f1( )  
{ auto int b;  
  ... f2( );  
  ...  
}  
void f2( )  
{ static int c;  
.....  
}
```

a的作用域

b的作用域

c的作用域

程序执行过程

main → f2 → main → f1 → f2 → f1 → main

a生存期

b生存期

c生存期

各种类型变量的作用域和存在性的情况

变量存储类别	函 数 内		函 数 外	
	作用域	存在性	作用域	存在性
自动变量和寄存器变量	√	√	×	×
静态局部变量	√	√	×	√
静态外部变量	√	√	√(只限本文件)	√
外部变量	√	√	√	√

(5) **static**对局部变量和全局变量的作用不同

- 局部变量使变量由动态存储方式改变为静态存储方式
- 全局变量使变量局部化(局部于本文件)，但仍为静态存储方式
- 从作用域角度看，凡有**static**声明的，其作用域都是局限的，或者是局限于本函数内(静态局部变量)，或者局限于本文件内(静态外部变量)

7.10 关于变量的声明和定义

52

- 函数的定义和声明的区别明显：
 - 函数的声明是函数的原型，放在声明部分中；
 - 函数的定义是函数本身，是独立的一个模块，不在声明部分中；
- 变量的定义和声明关系稍微复杂了一些。
 - 在声明部分出现的变量一种是需要建立存储空间的，称为定义性声明，或定义。而另一种是不需要建立存储空间的，称为引用性声明。（广义）
 - 通常，把建立存储空间的声明称为定义，而把不需要建立存储空间的声明称为声明（狭义）。
 - 外部变量只能定义一次，而声明可以有多次。

7.11 内部函数和外部函数

7.11.1 内部函数

7.11.2 外部函数

7.11.1 内部函数

- 如果一个函数只能被本文件中其他函数所调用，它称为**内部函数（静态函数）**。
- 在定义内部函数时，在函数名和函数类型的前面加**static**，即：
static 类型名 函数名(形参表)
 - 不同文件中即使有同名的内部函数也互不干扰。
- 通常把只能由本文件使用的函数和外部变量放在文件的开头，前面都冠以**static**使之局部化，其他文件不能引用，提高了程序的可靠性。

7.11.2 外部函数

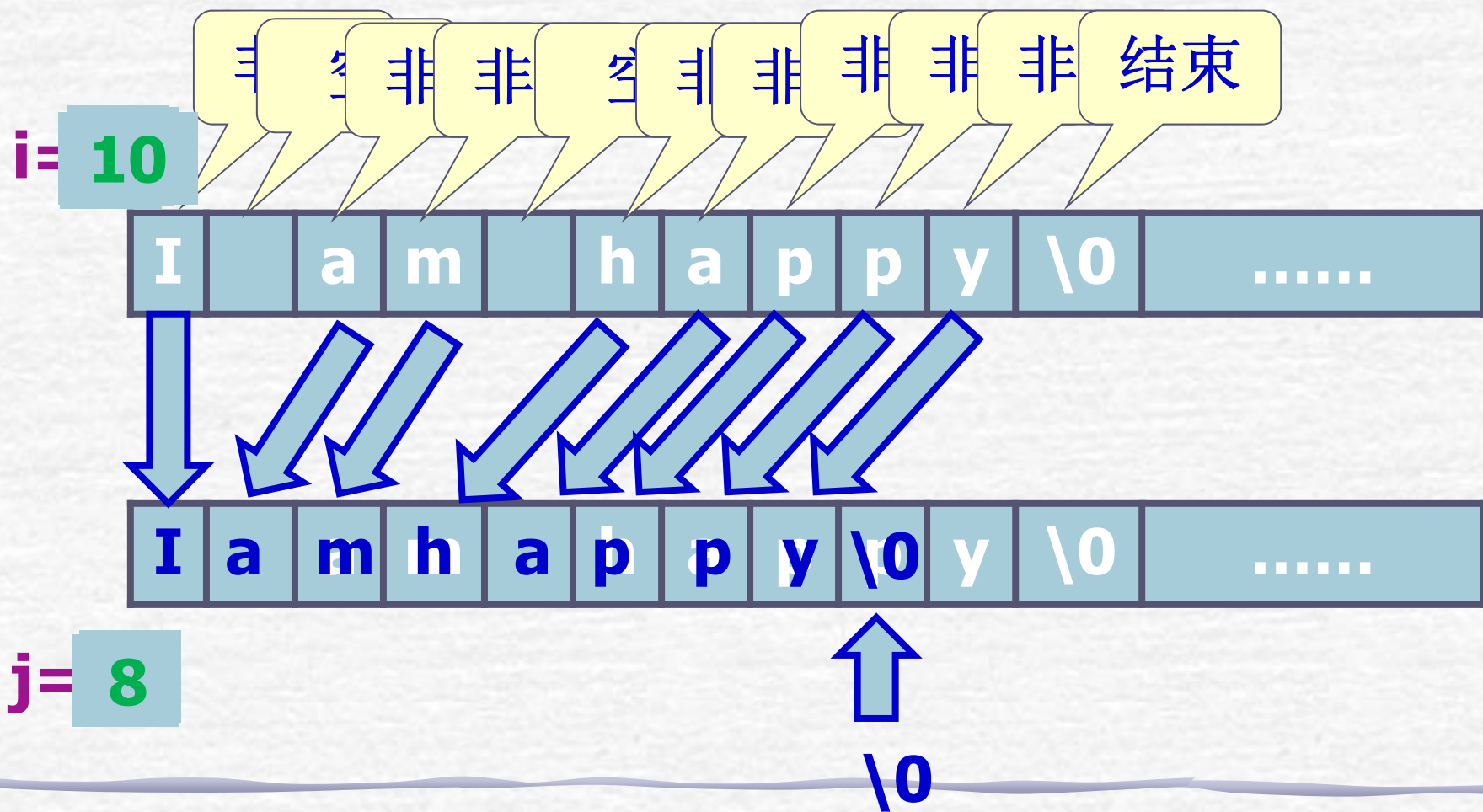
- 如果在定义函数时，在函数首部的最左端加关键字**extern**，则此函数是**外部函数**，可供其他文件调用。
- 如函数首部可以为
extern int fun (int a, int b)
- 如果在定义函数时省略**extern**，则默认为外部函数。
- 在需要调用外部函数的地方，用**extern**声明（即在函数原型基础上前加**extern**）所用的函数是外部函数。**C**语言允许省略**extern**，即只需使用函数原型。

【例7.20】 有一个字符串，内有若干个字符，今输入一个字符，要求程序将字符串中该字符删去。用外部函数实现。

解题思路：

- 分别定义3个函数用来输入字符串、删除字符、输出字符串
- 按题目要求把以上3个函数分别放在3个文件中。
main函数在另一文件中，**main**函数调用以上3个函数，实现题目的要求

删除空格的思路



file1 (文件1)

```
#include <stdio.h>
int main()
{
    extern void enter_string(char str[]);
    extern void delete_string(char str[],
                              char ch);
    extern void print_string(char str[]);
    char c, str[80];
    enter_string(str);
    scanf("%c", &c);
    delete_string(str, c);
    print_string(str);
    return 0;
}
```

声明在本函数中将要调用的已在其他文件中定义的3个函数

```
void enter_string(char str[80])
```

```
{ gets(str); }
```

file2 (文件2)

```
void delete_string(char str[],char ch)
```

```
{ int i,j;
```

```
  for(i=j=0;str[i]!='\0';i++)
```

```
    if(str[i]!=ch) str[j++]=str[i];
```

```
  str[j]='\0';
```

```
}
```

file3 (文件3)

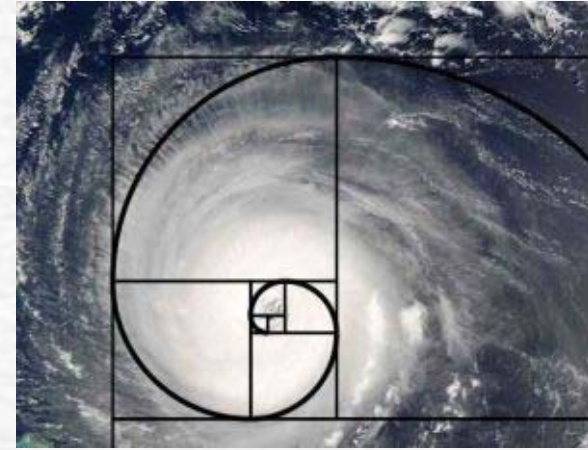
```
void print_string(char str[])
```

```
{ printf("%s\n",str); }
```

file4 (文件4)

自然界中的递归

60



Hyphaene Compressa - Doom Palm

© Shlomit Pinter

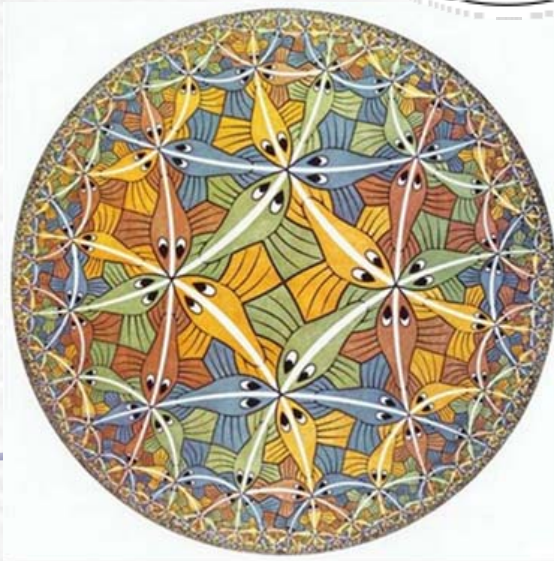
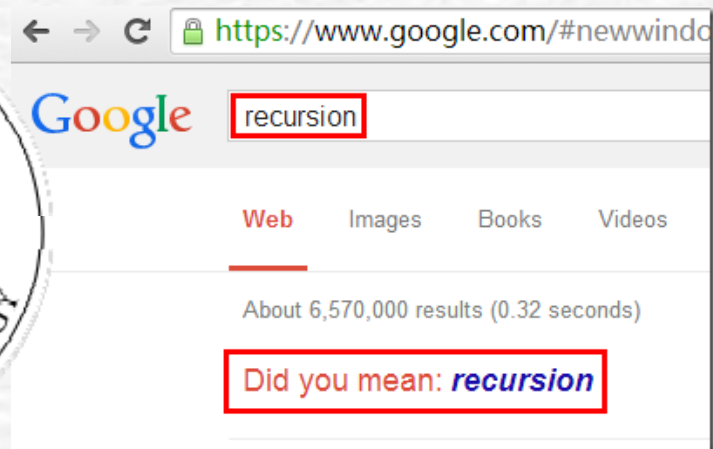
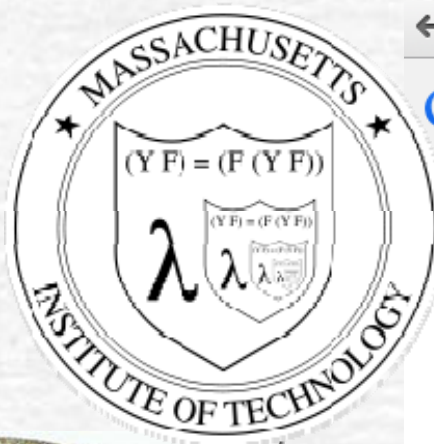
有趣的递归

61

从前有座山，山上有座庙，庙里有个老和尚在给小和尚讲故事：“从前有座山，山上有座庙，庙里有个老和尚在给小和尚讲故事：……”

你站在桥上看风景，看风景的人在楼上看你。明月装饰了你的窗子，你装饰了别人的梦。

——卞之琳《断章》



7.6 函数的递归调用

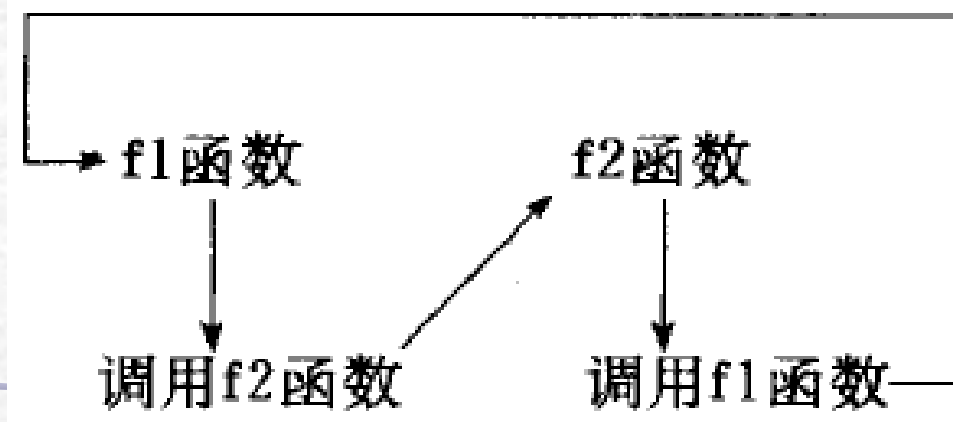
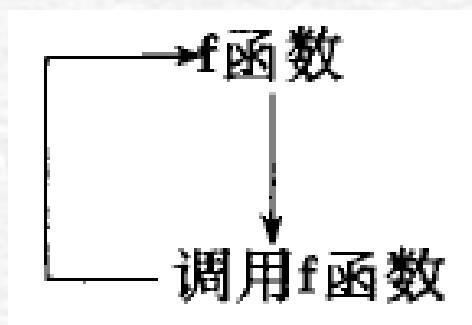
函数的递归调用 —— 在调用一个函数的过程中又出现**直接或间接地**调用函数本身。

两种情况：

1. **直接调用**：f1中调用f1；
2. **间接调用**：f1中调用f2，而f2再调用f1。

程序中出现的递归调用应当是**有限次数**的、**有终止**的递归调用，这通常用**if**语句来控制。

有限递归!



7.6 函数的递归调用

例7.6 有5个学生坐在一起

- 问第5个学生多少岁？他说比第4个学生大2岁
- 问第4个学生岁数，他说比第3个学生大2岁
- 问第3个学生，又说比第2个学生大2岁
- 问第2个学生，说比第1个学生大2岁
- 最后问第1个学生， he说是10岁
- 请问第5个学生多大

7.6 函数的递归调用

解题思路:

- 要求第 5 个年龄，就必须先知道第 4 个年龄
- 要求第 4 个年龄必须先知道第 3 个年龄
- 第 3 个年龄又取决于第 2 个年龄
- 第 2 个年龄取决于第 1 个年龄
- 每个学生年龄都比其前 1 个学生的年龄大 2

7.6 函数的递归调用

解题思路：

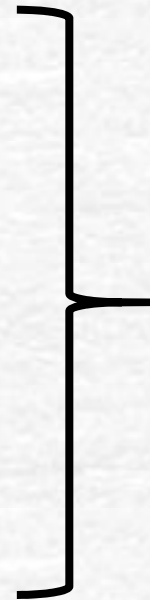
$$\text{age}(5) = \text{age}(4) + 2$$

$$\text{age}(4) = \text{age}(3) + 2$$

$$\text{age}(3) = \text{age}(2) + 2$$

$$\text{age}(2) = \text{age}(1) + 2$$

$$\text{age}(1) = 10$$



$$\text{age}(n) = 10 \quad (n = 1)$$

$$\text{age}(n) = \text{age}(n - 1) + 2 \quad (n > 1)$$

$$\begin{array}{l} \text{age}(5) \\ \hline = \text{age}(4) + 2 \end{array}$$

$$\begin{array}{l} \text{age}(4) \\ \hline = \text{age}(3) + 2 \end{array}$$

$$\begin{array}{l} \text{age}(3) \\ \hline = \text{age}(2) + 2 \end{array}$$

$$\begin{array}{l} \text{age}(2) \\ \hline = \text{age}(1) + 2 \end{array}$$

回溯阶段

$$\begin{array}{l} \text{age}(1) \\ \hline = 10 \end{array}$$

$$\begin{array}{l} \text{age}(5) \\ \hline = 18 \end{array}$$

$$\begin{array}{l} \text{age}(4) \\ \hline = 16 \end{array}$$

$$\begin{array}{l} \text{age}(3) \\ \hline = 14 \end{array}$$

$$\begin{array}{l} \text{age}(2) \\ \hline = 12 \end{array}$$

递推阶段

$$\begin{array}{l} \text{age}(5) \\ \hline = \text{age}(4) + 2 \end{array}$$

$$\begin{array}{l} \text{age}(4) \\ \hline = \text{age}(3) + 2 \end{array}$$

$$\begin{array}{l} \text{age}(3) \\ \hline = \text{age}(2) + 2 \end{array}$$

$$\begin{array}{l} \text{age}(1) \\ \hline = \text{age}(1) + 2 \end{array}$$

回溯阶段

结束递归的条件

$$\begin{array}{l} \text{age}(1) \\ \hline = 10 \end{array}$$

$$\begin{array}{l} \text{age}(5) \\ \hline = 18 \end{array}$$

$$\begin{array}{l} \text{age}(4) \\ \hline = 16 \end{array}$$

$$\begin{array}{l} \text{age}(3) \\ \hline = 14 \end{array}$$

$$\begin{array}{l} \text{age}(2) \\ \hline = 12 \end{array}$$

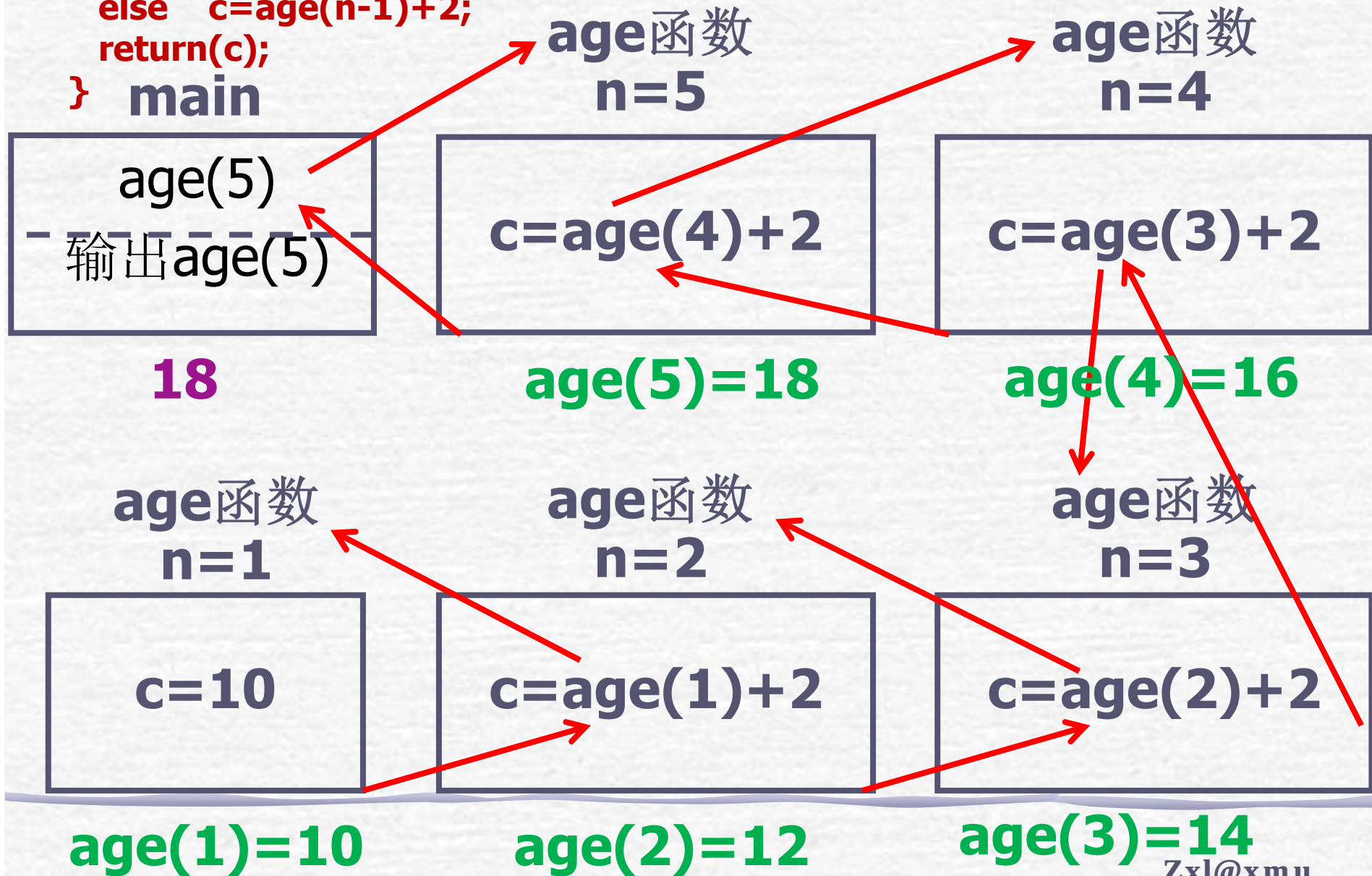
递推阶段

```
#include <stdio.h>
int main()
{ int age(int n);
  printf("NO.5,age:%d\n",age(5));
  return 0;
}
int age(int n)
{ int c;
  if(n==1) c=10;
  else c=age(n-1)+2;
  return(c);
}
```

```
NO.5,age:18
```



```
int age(int n)
{ int c;
  if(n==1) c=10;
  else c=age(n-1)+2;
  return(c);
} main
```



7.6 函数的递归调用

设计递归算法时，可先写出问题的递归定义（同第二数学归纳法类似）：

1. **基本项（递归出口）**：描述了一个或几个问题的终结状态（不需要再继续递归就可以直接求解的状态）。
 - 任何实际应用的递归过程，除错误情况外，必定能经过有限层次的递归而终止。
2. **归纳项**：描述了如何实现从当前状态到终结状态的转化，即描述了这类原问题和子问题之间的转化关系。（**向着递归出口的方向转化**）

例7.7 用递归方法求 $n!$ 。

解题思路：

- 求 $n!$ 可以用递推方法：即从 1 开始，乘 2，再乘 3 一直乘到 n 。
- 递推法的特点是从一个已知的事实(如 $1!=1$)出发，按一定规律推出下一个事实(如 $2!=1!*2$)，再从这个新的已知的事实出发，再向下推出一个新的新的事实($3!=3*2!$)。 $n!=n*(n-1)!$ 。

例7.7 用递归方法求 $n!$ 。

解题思路：

- 求 $n!$ 也可以用递归方法，即 $5!$ 等于 $4! \times 5$ ，而 $4! = 3! \times 4 \dots$ ， $1! = 1$
- 可用下面的递归公式表示：

$$n! = \begin{cases} n! = 1 & (n = 0, 1) \\ n \bullet (n-1)! & (n > 1) \end{cases}$$

```
#include <stdio.h>
```

```
int main()
```

```
{int fac(int n);
```

```
int n; int y;
```

```
printf("input an integer number:");
```

```
scanf("%d",&n);
```

```
y=fac(n);
```

```
printf("%d!=%d\n",n,y);
```

```
return 0;
```

```
}
```

```
int fac(int n)
```

```
{
```

```
    int f;
```

```
    if(n<0)
```

```
        printf("n<0,data error!");
```

```
    else if(n==0 || n==1)
```

```
        f=1;
```

```
    else
```

```
        f=fac(n-1)*n;
```

```
    return(f);
```

```
}
```

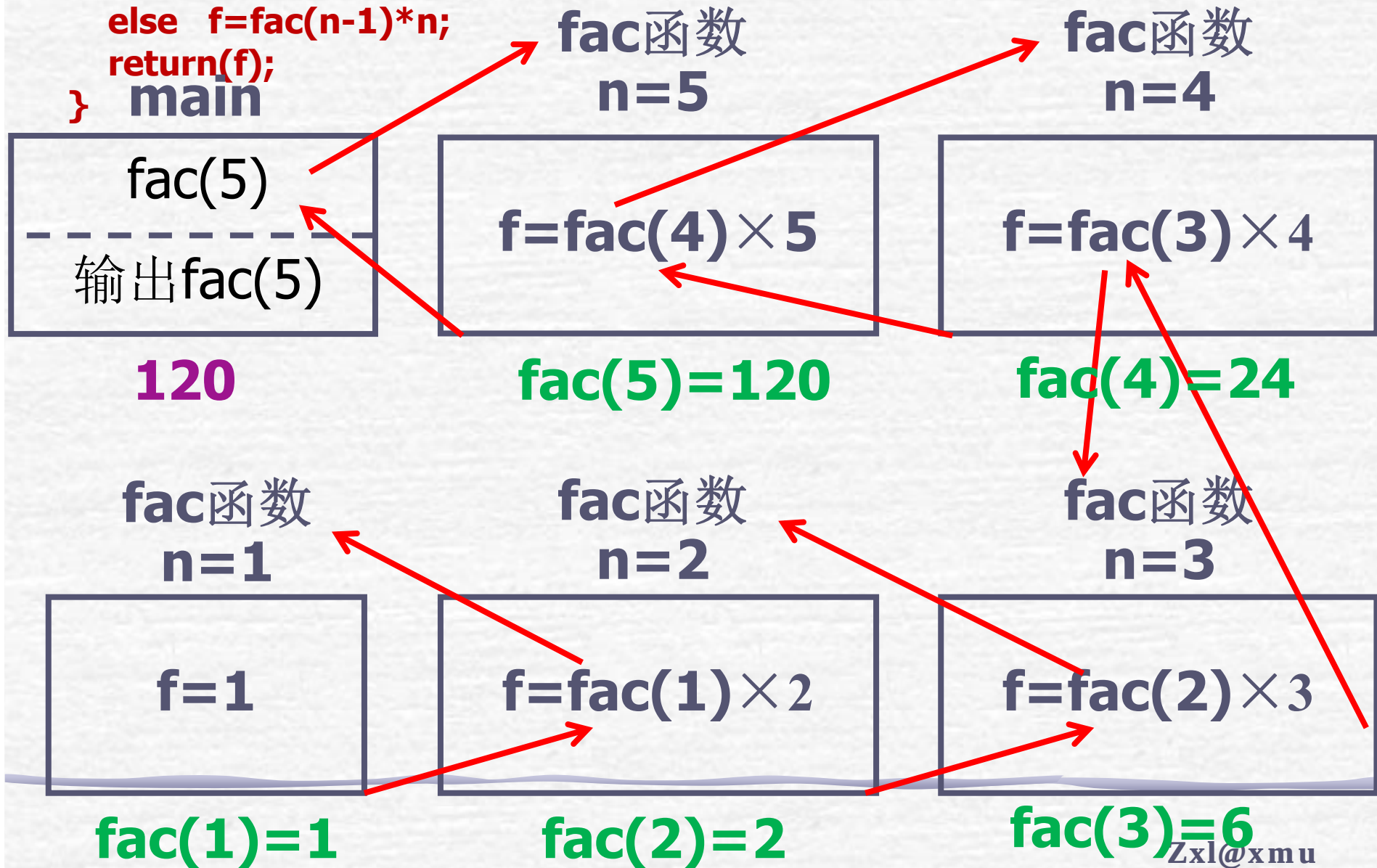
$$n! = \begin{cases} n! = 1 & (n = 0, 1) \\ n \bullet (n-1)! & (n > 1) \end{cases}$$

注意溢出

```
input an integer number:31
31!=738197504
```



```
int fac(int n)
{ int f;
  if(n<0) printf("n<0,data error!");
  else if(n==0 || n==1) f=1;
  else f=fac(n-1)*n;
  return(f);
} main
```



7.6 函数的递归调用 (2)

【例1】猴子吃桃问题（P138 5.12）。

递推公式：
$$\begin{cases} f_{i+1} = f_i / 2 - 1 \rightarrow f_i = 2 * (f_{i+1} + 1) \\ f_{10} = 1 \end{cases}$$

解法一（循环）：

```
main()
```

```
{ int i, f=1;
```

```
  for(i=9; i>0; i--) f = 2 * (f + 1);
```

```
  printf("%d\n",f);
```

```
}
```

7.6 函数的递归调用 (3)

解法二（递归）：

```
int f1(int n) /* 求第n天有几个桃子 */
{
    if (n==10)
        return 1; // 递归调用的终止出口
    else
        return 2*(f1(n+1)+1);
}
main()
{ int i,f;
  f = f1(1);
  printf("%d\n",f);
}
```

$$\begin{cases} f_i = 2 * (f_{i+1} + 1) \\ f_{10} = 1 \end{cases}$$

解法一（循环）：

```
main()
{
    int i, f=1;
    for(i=9;i>0;i--) f=2*(f+1);
    printf("%d\n",f);
}
```


改变函数语句的顺序会导致执行结果的巨大变化

```

1.#include <stdio.h>
2.void func(int a)
3.{
4.    if (a < 4)
5.    {
6.        printf("%d\n", a);
7.        func(a+1);
8.    }
9.}
10.int main()
11.{
12.    func(0);
13.    return 0;
14.}

```

0
1
2
3

```

1.#include <stdio.h>
2.void func(int a)
3.{
4.    if (a < 4)
5.    {
6.        func(a+1);
7.        printf("%d\n", a);
8.    }
9.}
10.int main()
11.{
12.    func(0);
13.    return 0;
14.}

```

3
2
1
0

递归的时空开销交待，递归层次有限，应避免过度使用递归。（尤其是当问题存在简便的非递归求解方法时）

汉诺塔（梵塔）：世界末日传说

法国数学家爱德华·卢卡斯曾编写过一个印度的古老传说：

- 在世界中心贝拿勒斯（在印度北部）的圣庙里，一块黄铜板上插着三根宝石针。印度教的主神梵天在创造世界的时候，在其中一根针上从下到上地穿好了由大到小的**64**片金片，这就是所谓的汉诺塔。
- 不论白天黑夜，总有一个僧侣在按照下面的法则移动这些金片：一次只移动一片，不管在哪根针上，小片必须在大片上面。
- 僧侣们预言，当所有的金片都从梵天穿好的那根针上移到另外一根针上时，世界就将在一声霹雳中消灭，而梵塔、庙宇和众生也都将同归于尽。



【数学论证】

假设有 n 片，移动次数是 $f(n)$ 显然 $f(1)=1$, $f(2)=3$, $f(3)=7$, 且 $f(k+1)=2*f(k)+1$ 。此后不难证明 $f(n)=2^n-1$ 。

- 当 $n=64$ 时，假如每秒钟一次，共需花费18446744073709551615秒，即移完这些金片需要5845.54亿年以上。
- 地球存在至今不过45亿年，太阳系的预期寿命据说也就是数百亿年。

例7.8 Hanoi（汉诺）塔问题。古代有一个梵塔，塔内有3个座A、B、C，开始时A座上有64个盘子，盘子大小不等，大的在下，小的在上。有一个老和尚想把这64个盘子从A座移到C座，但规定每次只允许移动一个盘，且在移动过程中在3个座上都始终保持大盘在下，小盘在上。在移动过程中可以利用B座。要求编程序输出移动一盘子的步骤。

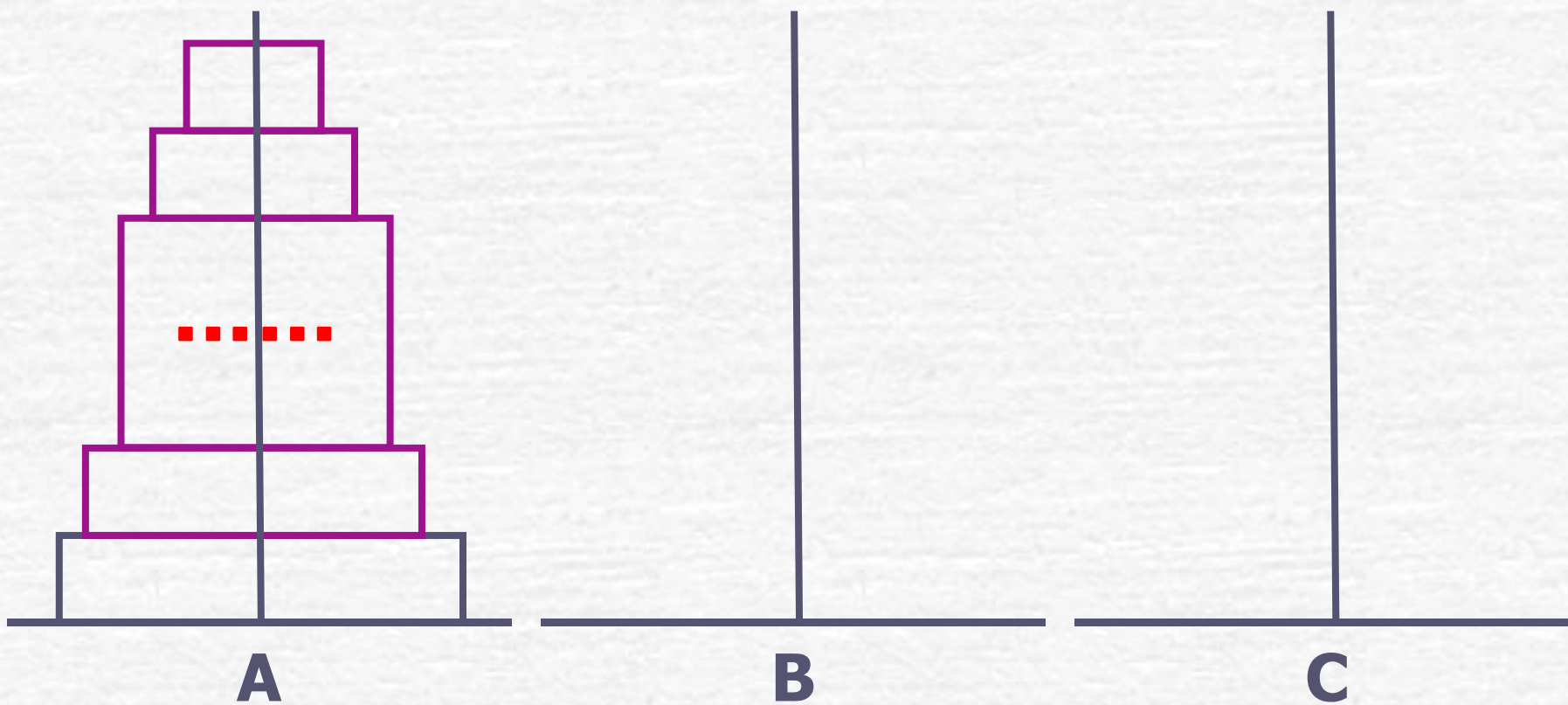


■ 解题思路:

- 要把**64**个盘子从**A**座移动到**C**座，需要移动大约 2^{64} 次盘子。一般人是不可能直接确定移动盘子的每一个具体步骤的
- 老和尚会这样想：假如有另外一个和尚能有办法将上面**63**个盘子从一个座移到另一座。那么，问题就解决了。此时老和尚只需这样做：
 - (1) 命令第**2**个和尚将**63**个盘子从**A**座移到**B**座；
 - (2) 自己将**1**个盘子（最底下的、最大的盘子）从**A**座移到**C**座；
 - (3) 再命令第**2**个和尚将**63**个盘子从**B**座移到**C**座。

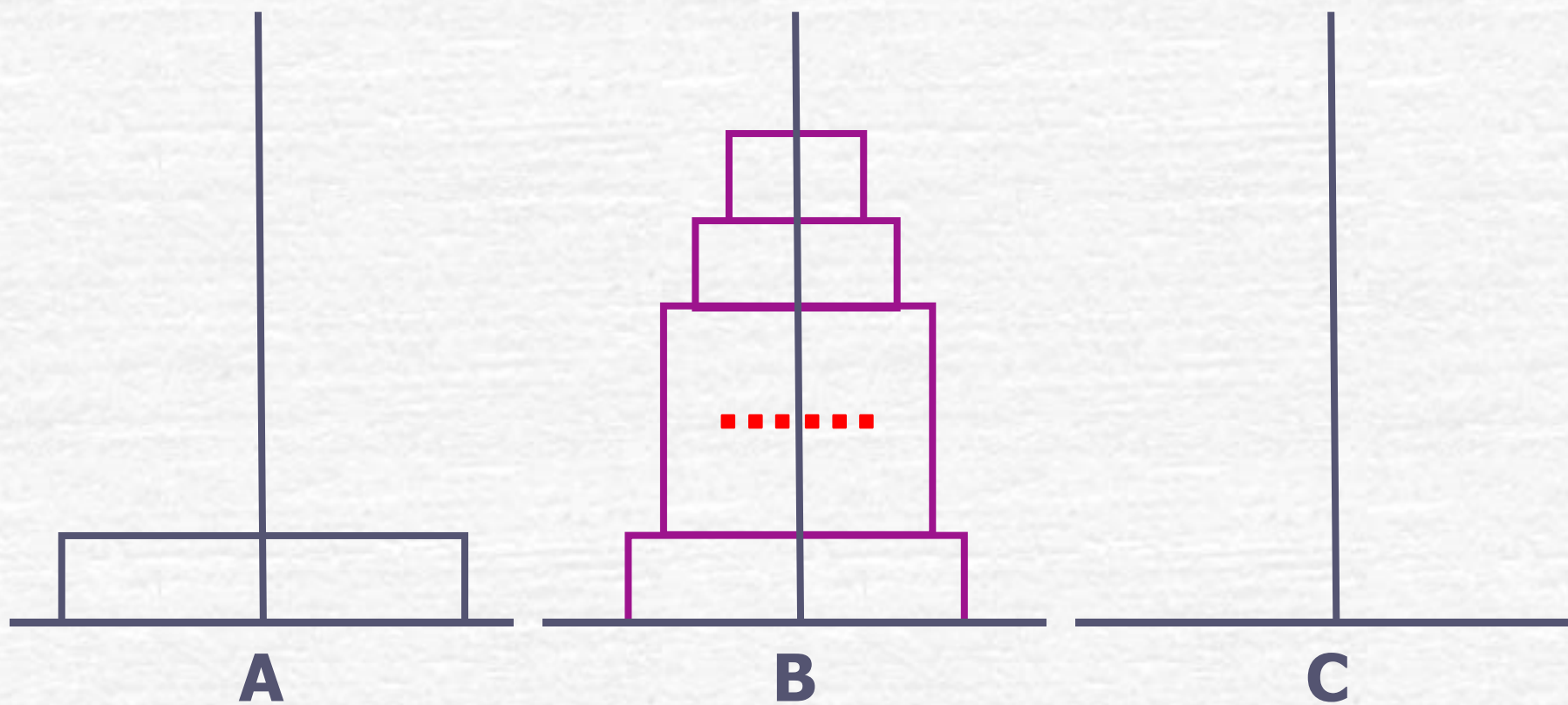
第1个和尚的做法

将**63**个从**A**到**B**



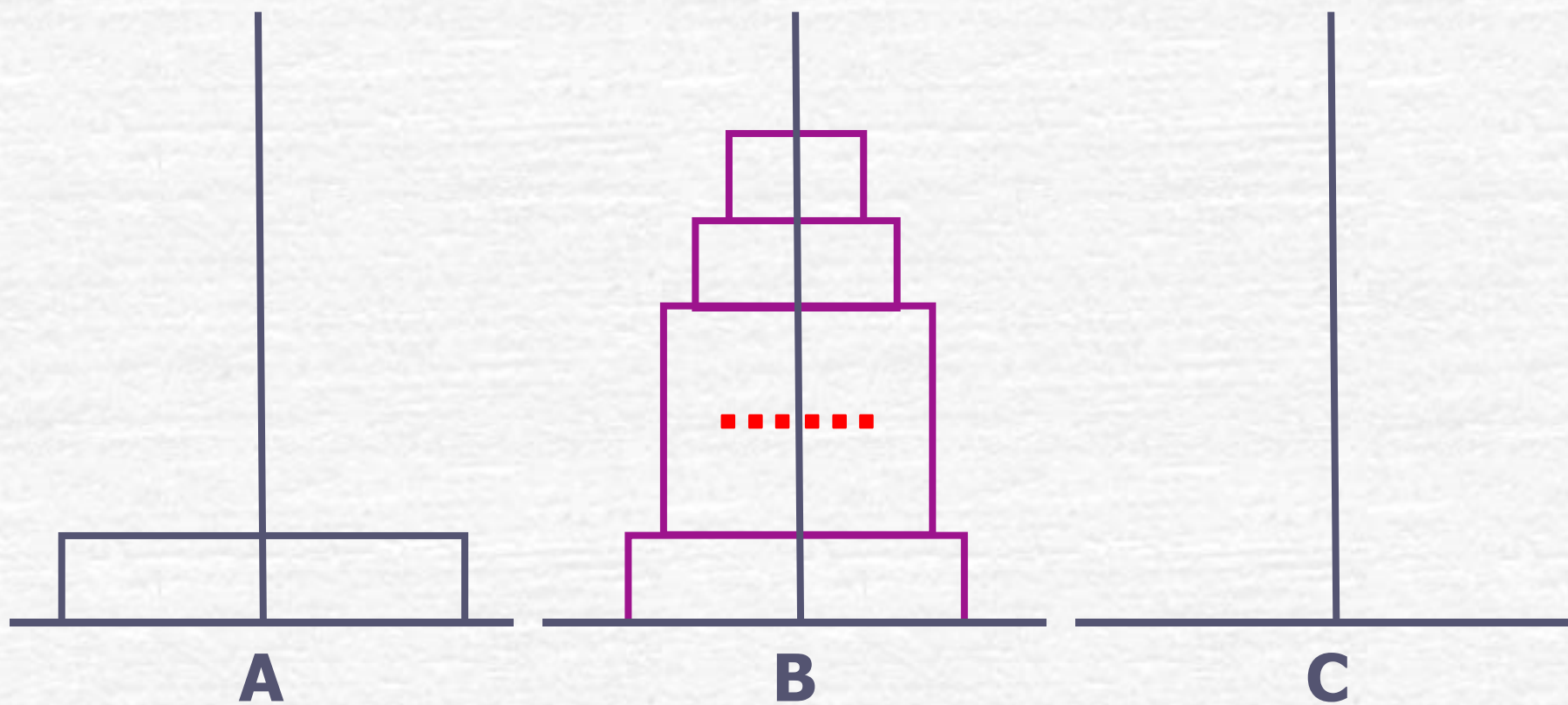
第**1**个和尚的做法

将**63**个从**A**到**B**



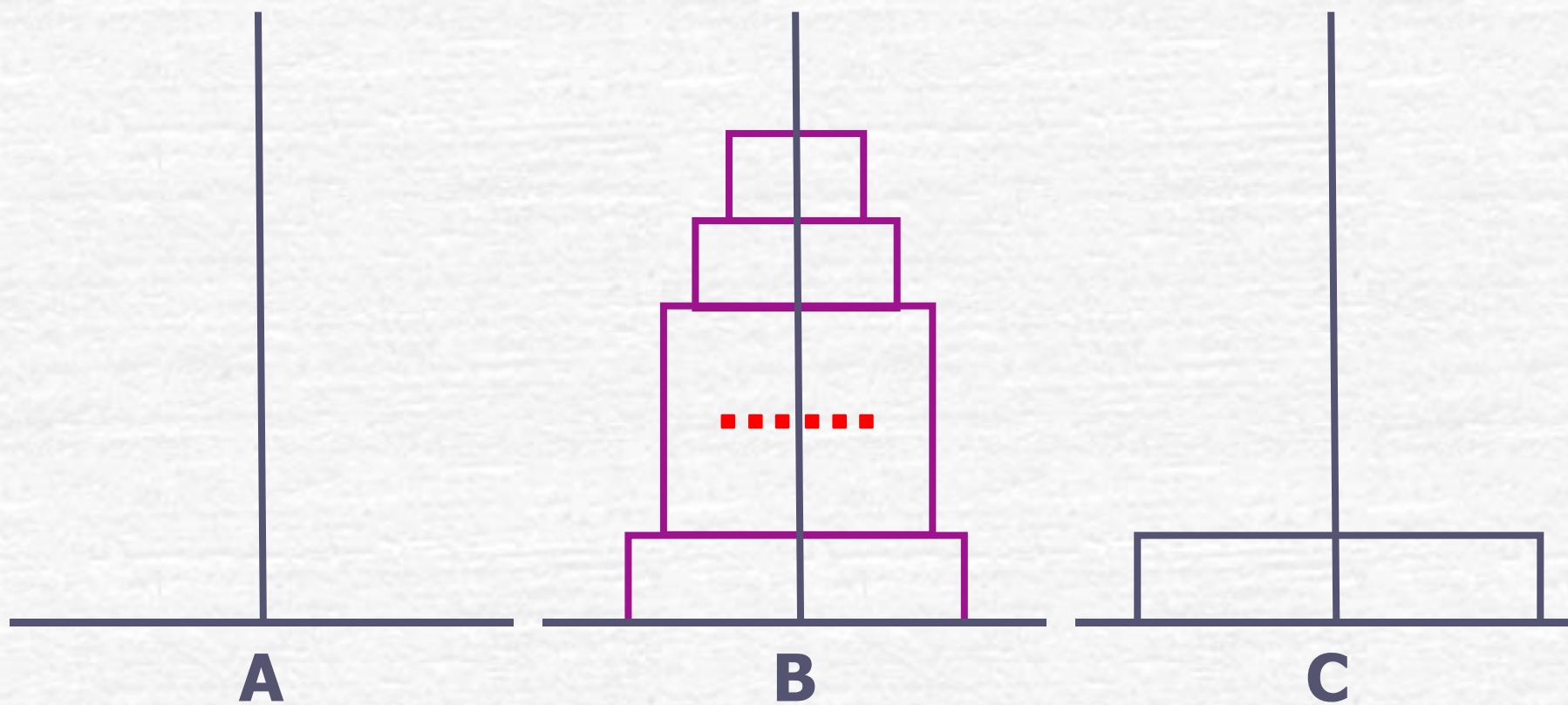
第**1**个和尚的做法

将**1**个从**A**到**C**



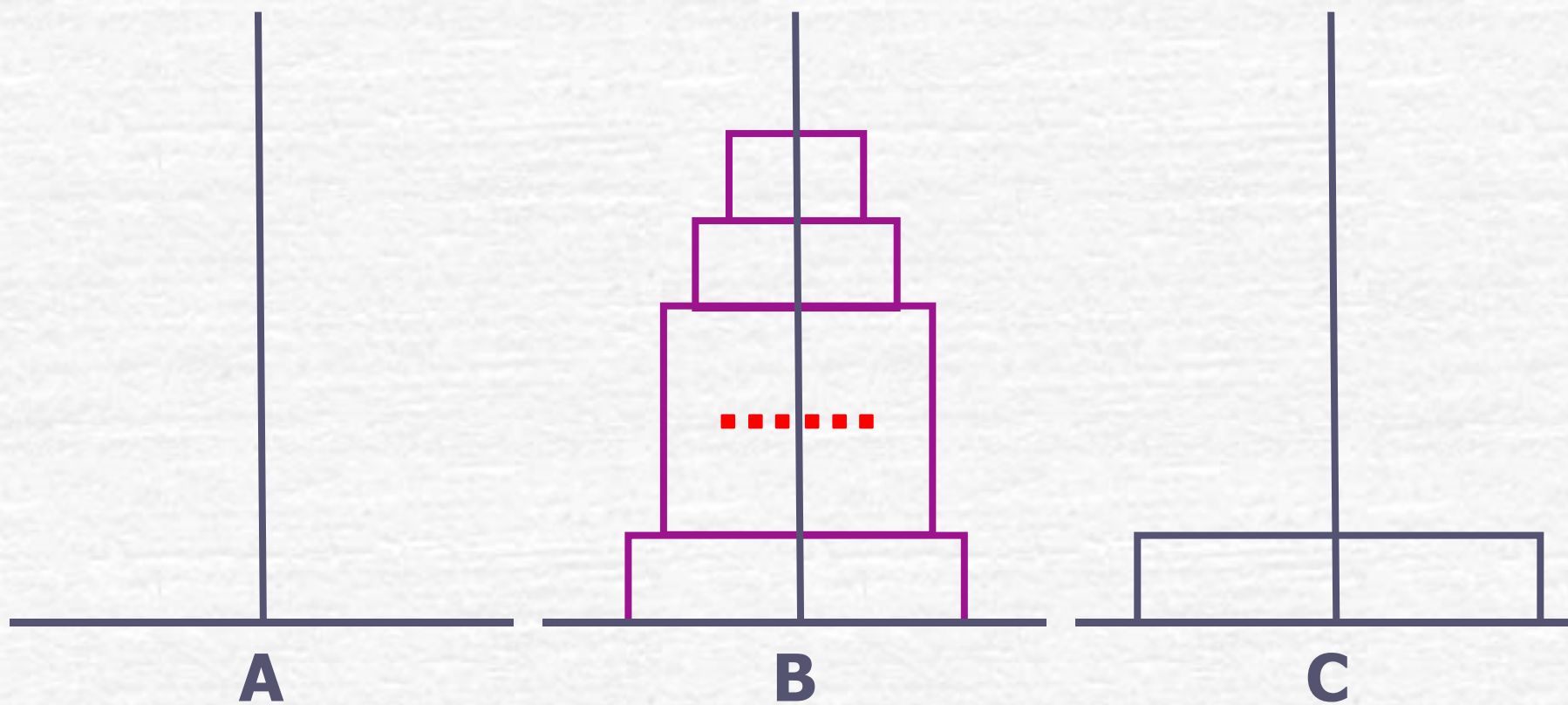
第**1**个和尚的做法

将**1**个从**A**到**C**



第**1**个和尚的做法

将**63**个从**B**到**C**



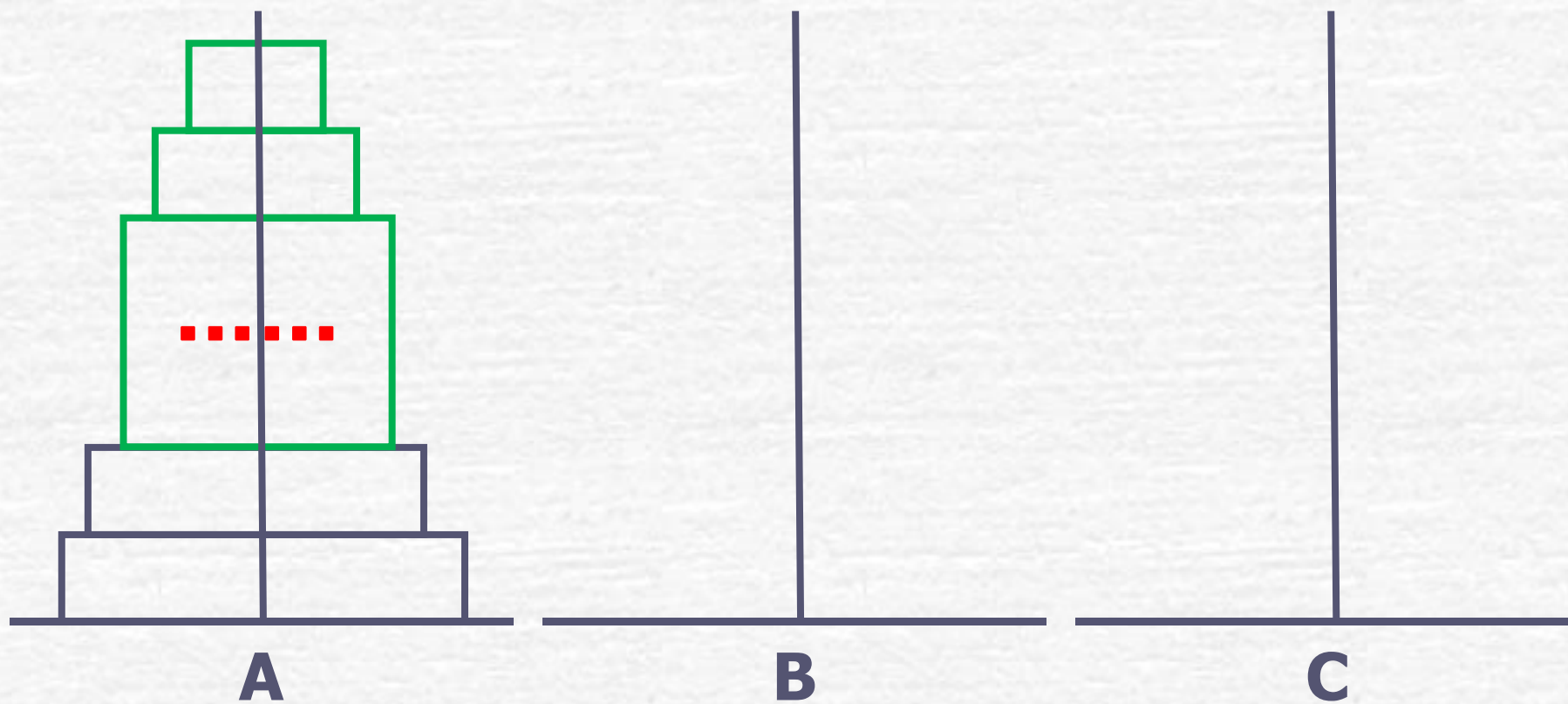
第**1**个和尚的做法

将**63**个从**B**到**C**



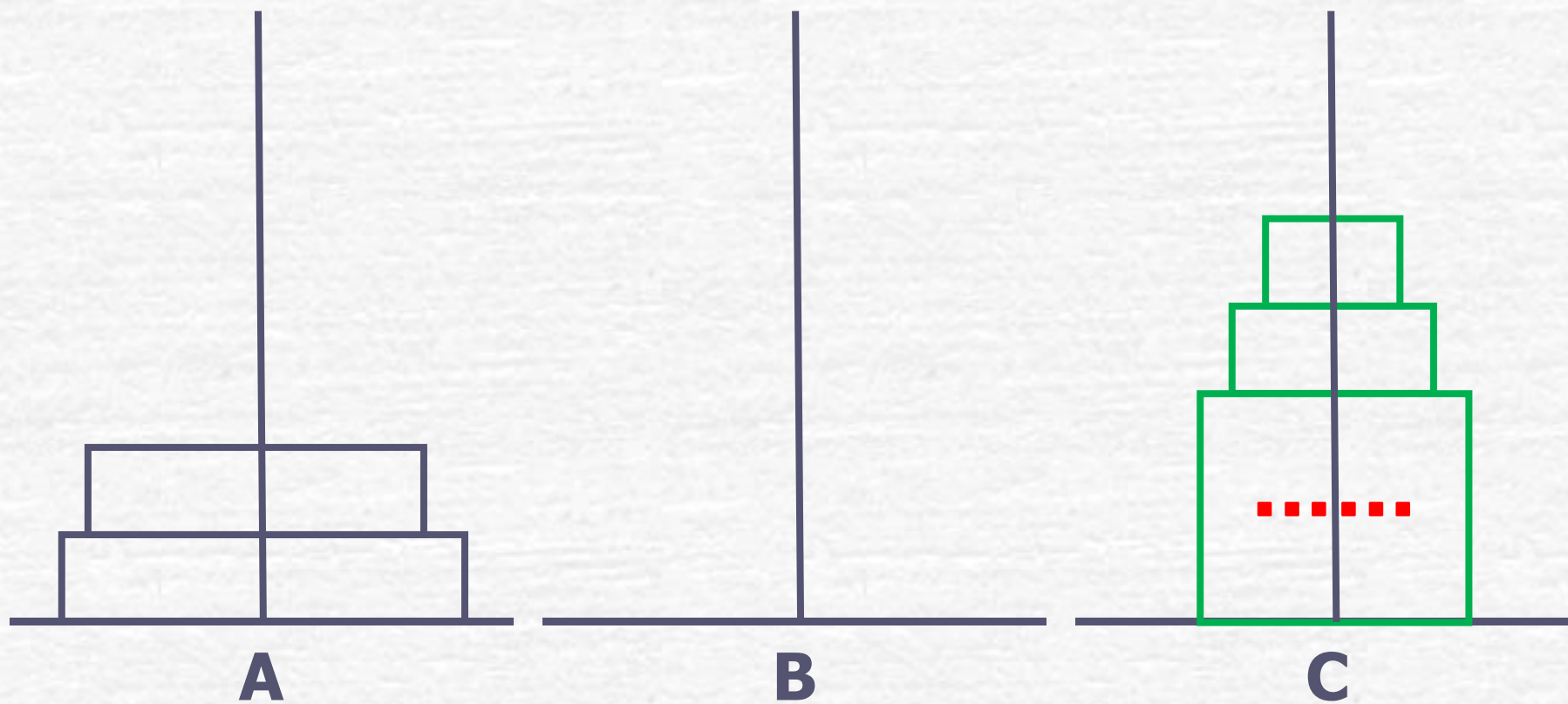
第2个和尚的做法

将**62**个从**A**到**C**



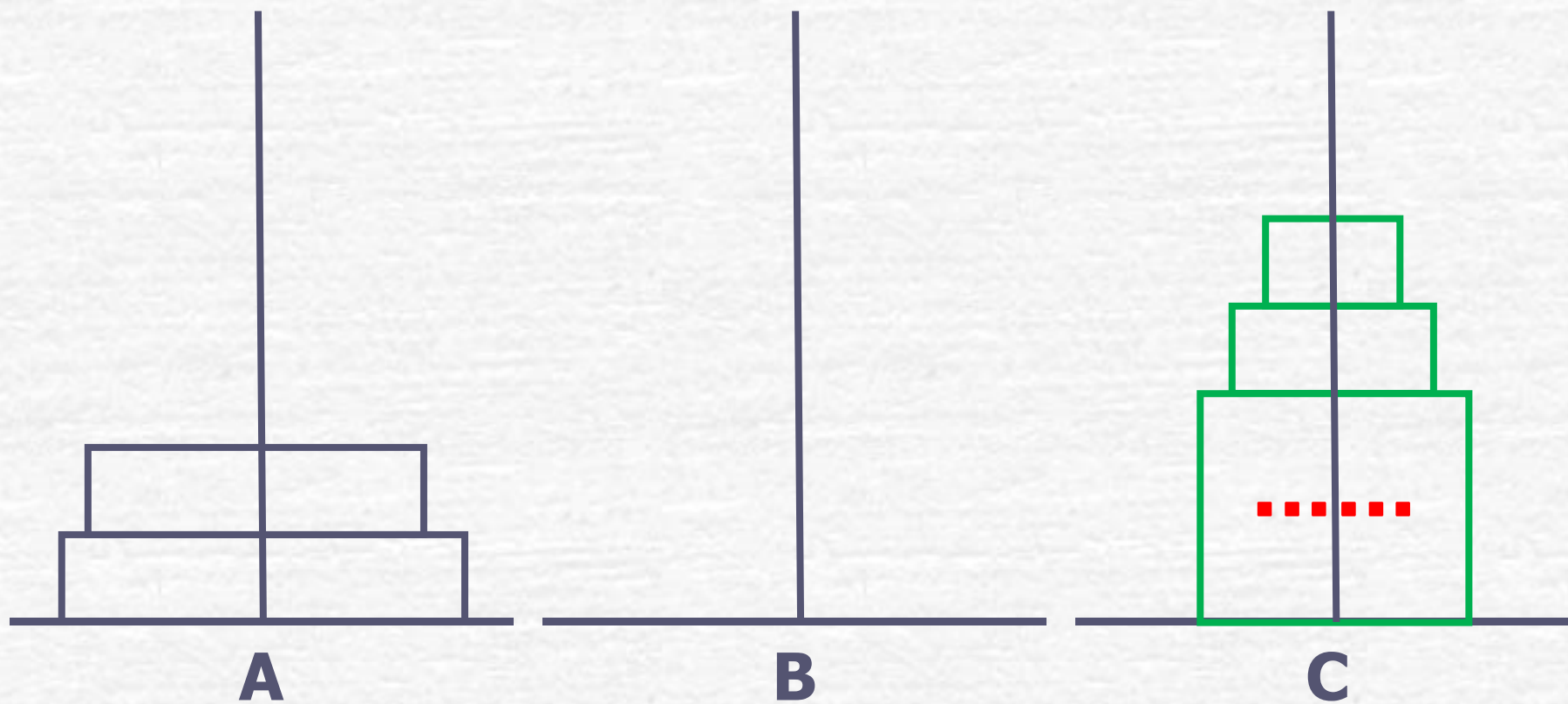
第2个和尚的做法

将**62**个从**A**到**C**



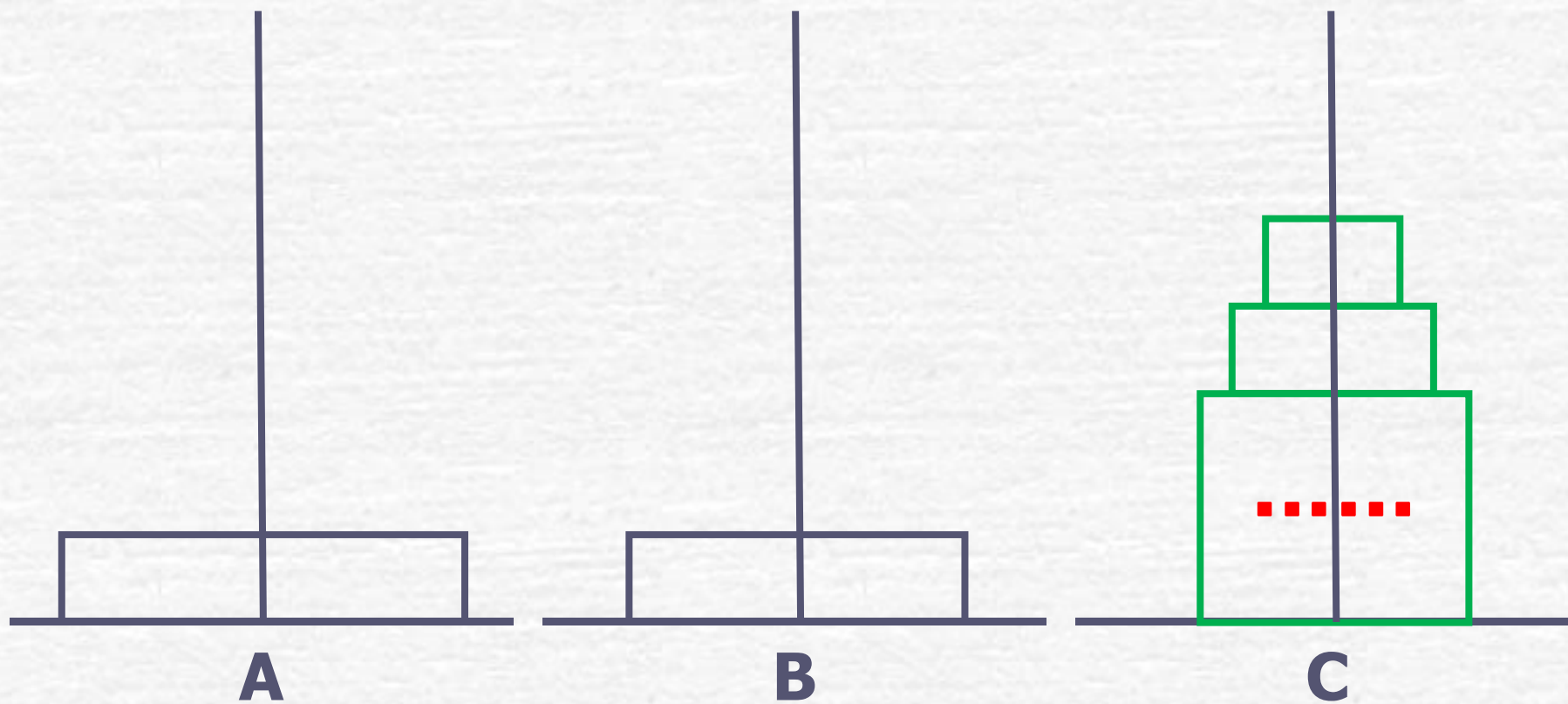
第2个和尚的做法

将**1**个从**A**到**B**



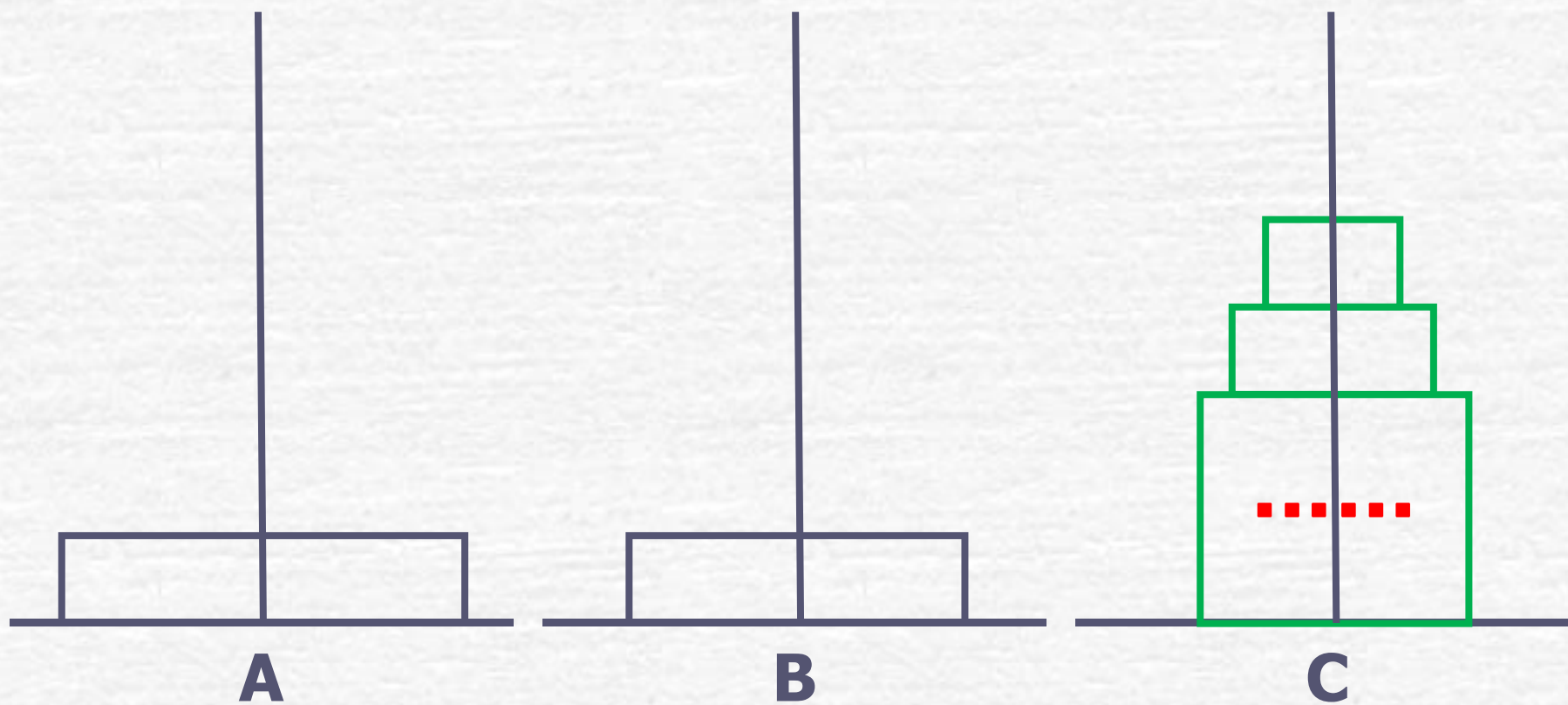
第2个和尚的做法

将**1**个从**A**到**B**



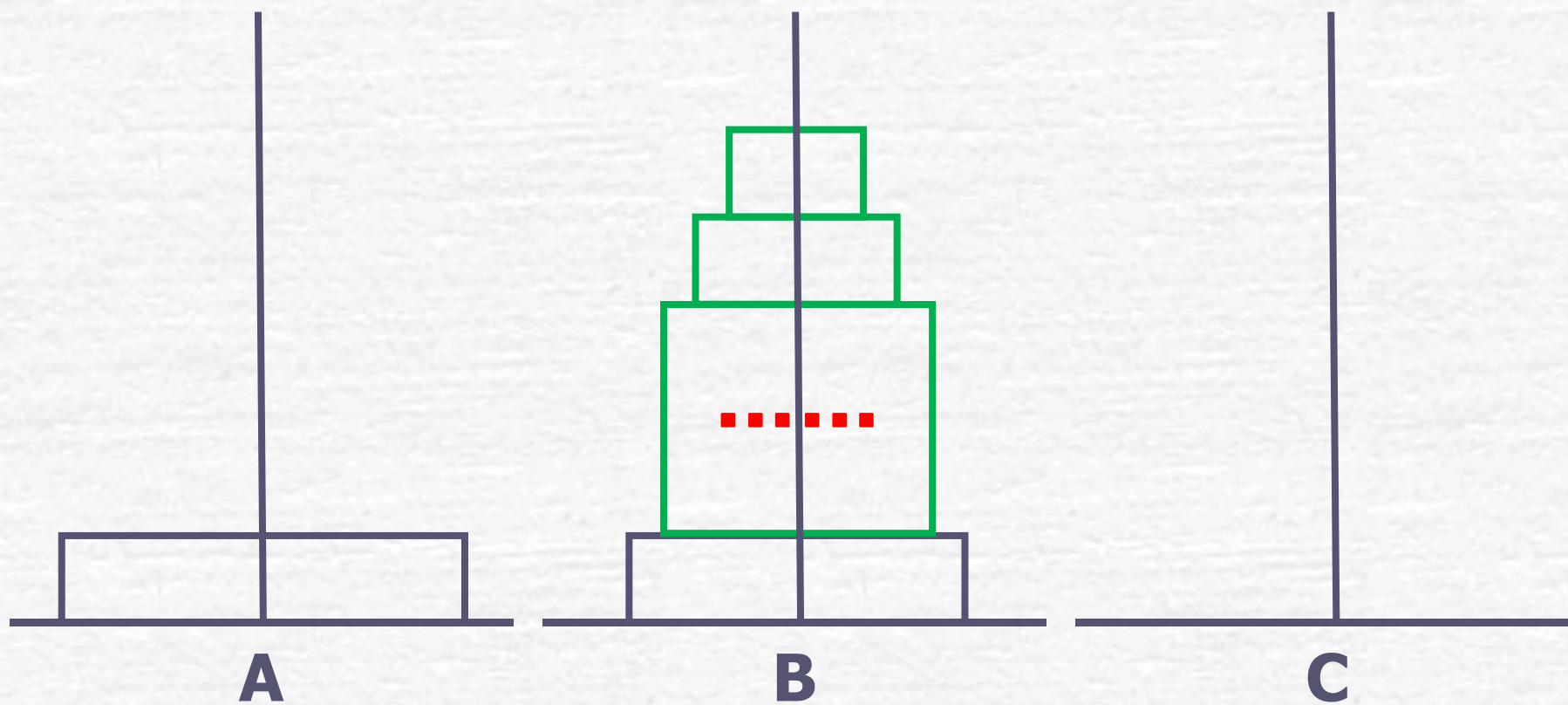
第2个和尚的做法

将62个从C到B



第2个和尚的做法

将62个从C到B



第**3**个和尚的做法

第**4**个和尚的做法

第**5**个和尚的做法

第**6**个和尚的做法

第**7**个和尚的做法

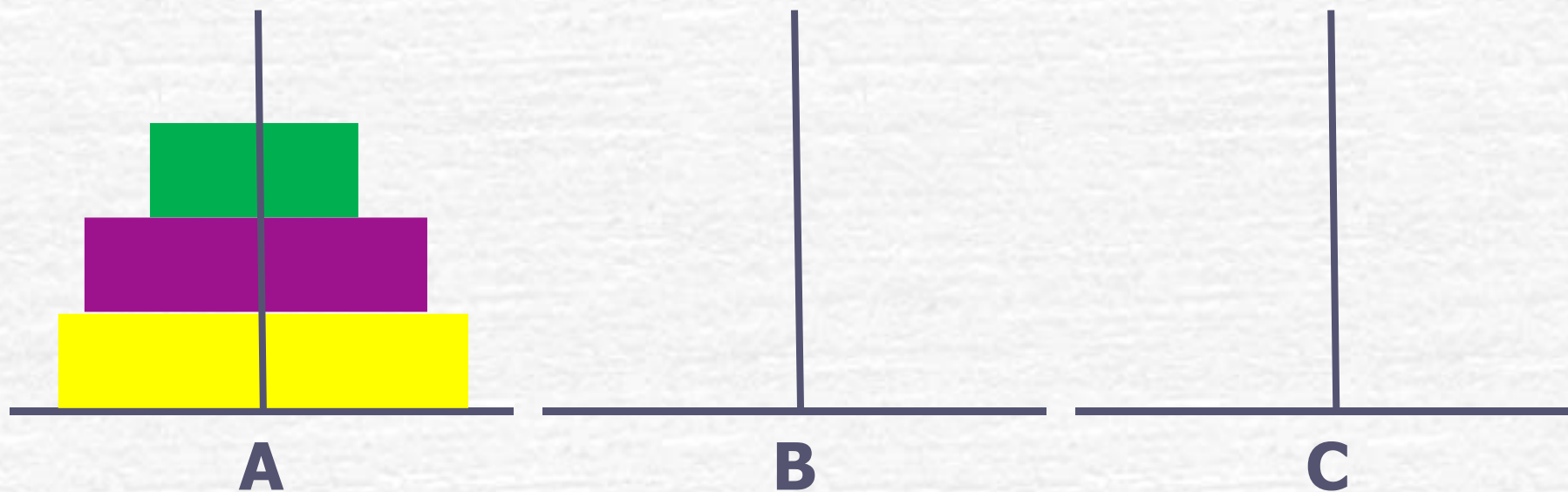
.....

第**63**个和尚的做法

第**64**个和尚仅做：将**1**个从**A**移到**C**

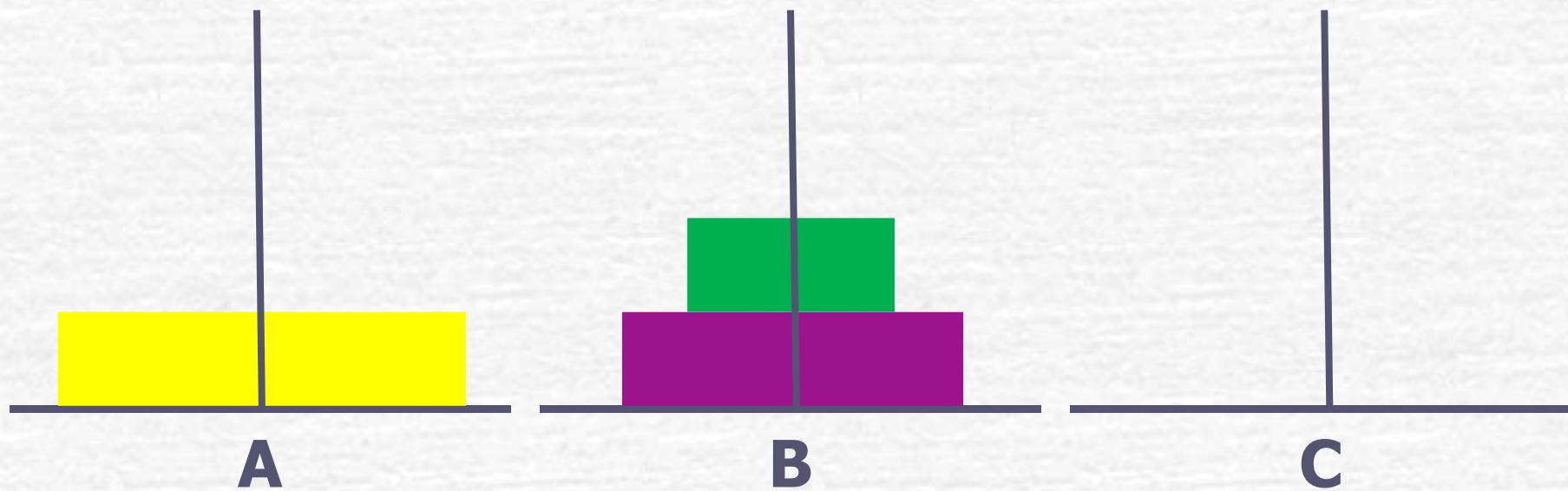
将**3**个盘子从**A**移到**C**的全过程

将**2**个盘子从**A**移到**B**



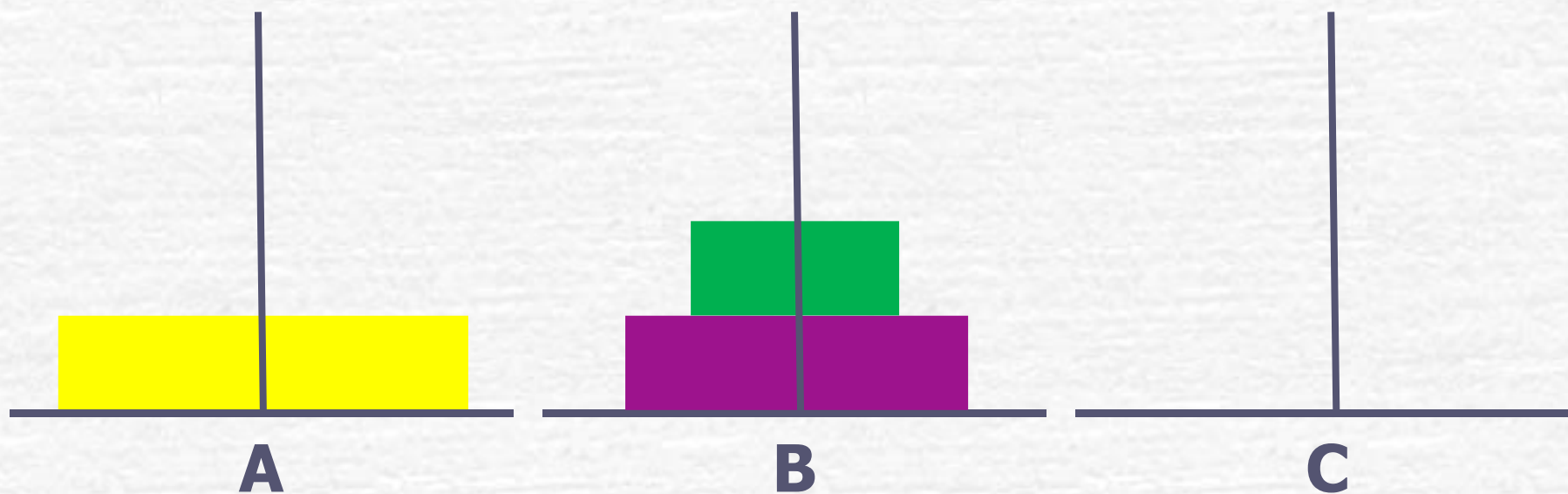
将**3**个盘子从**A**移到**C**的全过程

将**2**个盘子从**A**移到**B**



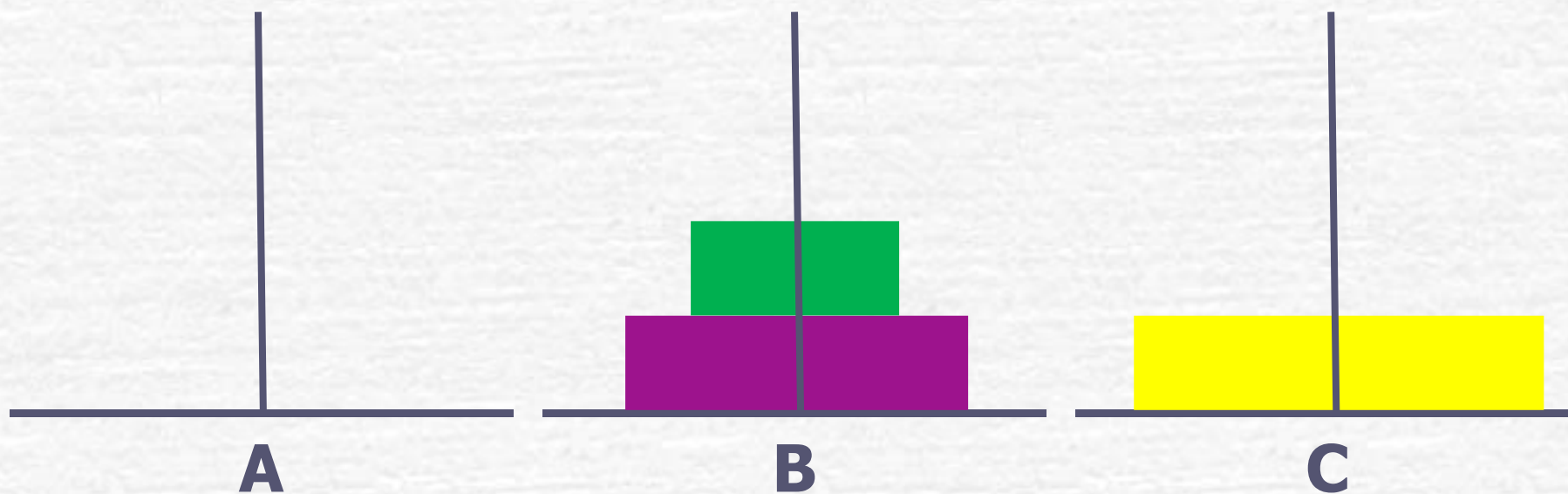
将**3**个盘子从**A**移到**C**的全过程

将**1**个盘子从**A**移到**C**



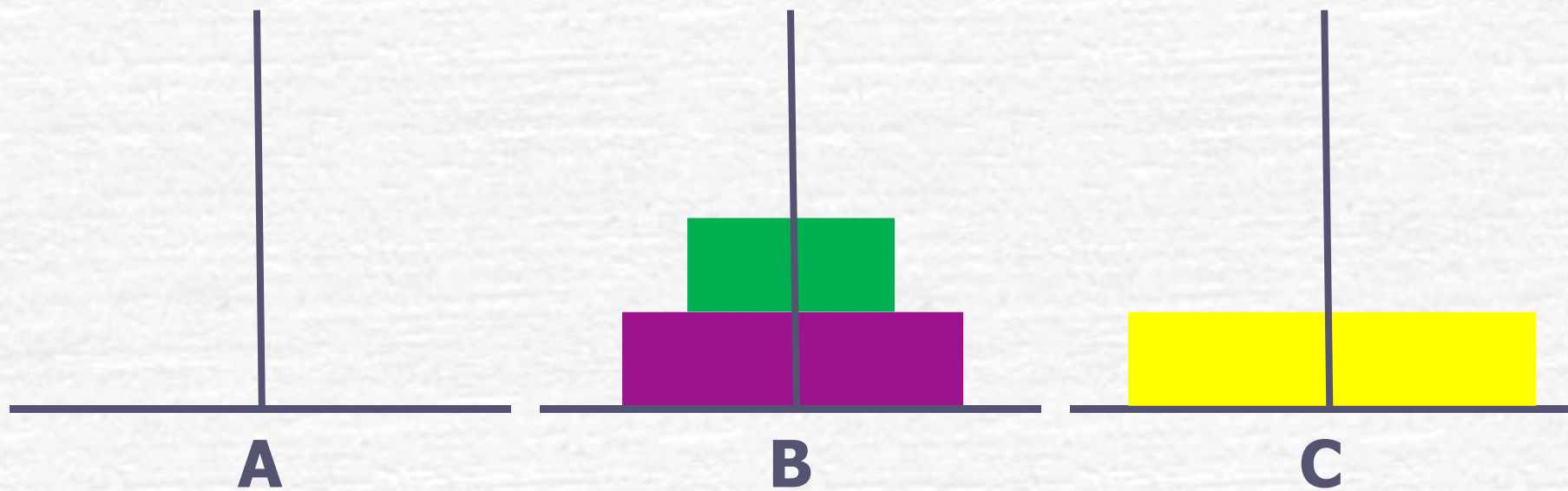
将**3**个盘子从**A**移到**C**的全过程

将**1**个盘子从**A**移到**C**



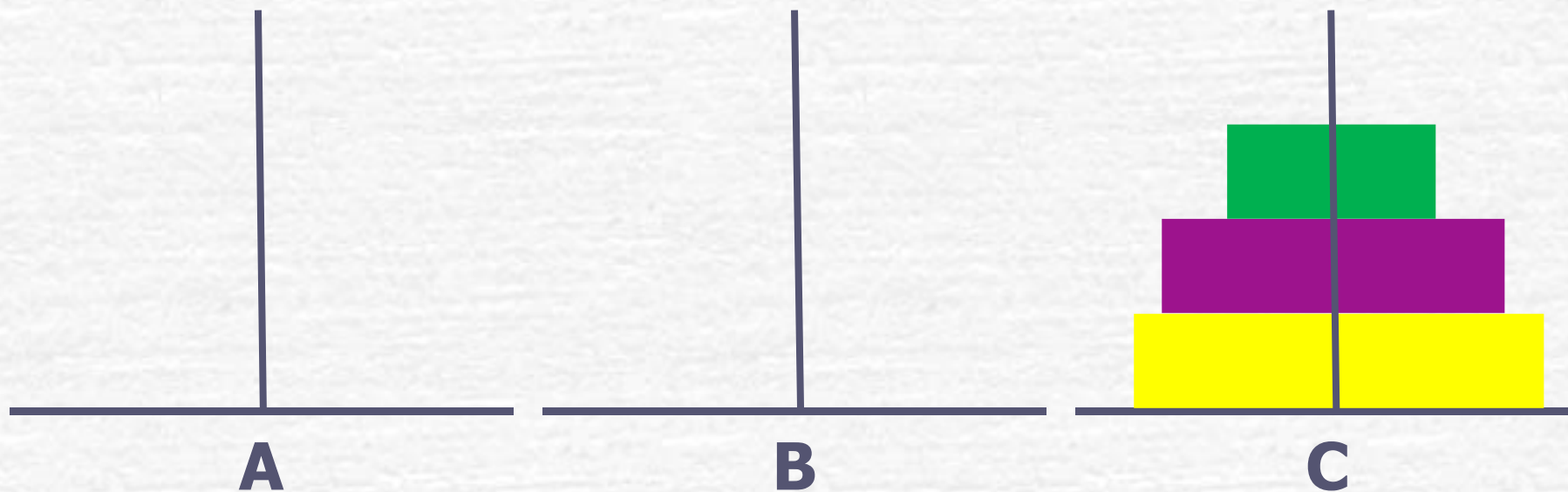
将**3**个盘子从**A**移到**C**的全过程

将**2**个盘子从**B**移到**C**



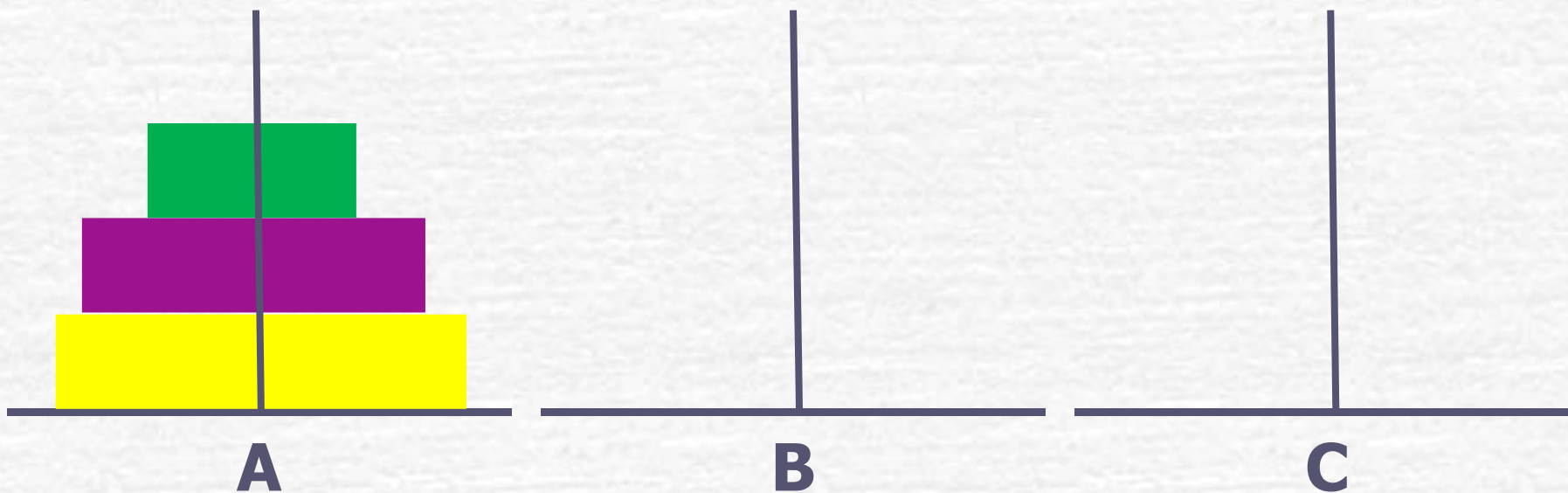
将**3**个盘子从**A**移到**C**的全过程

将**2**个盘子从**B**移到**C**



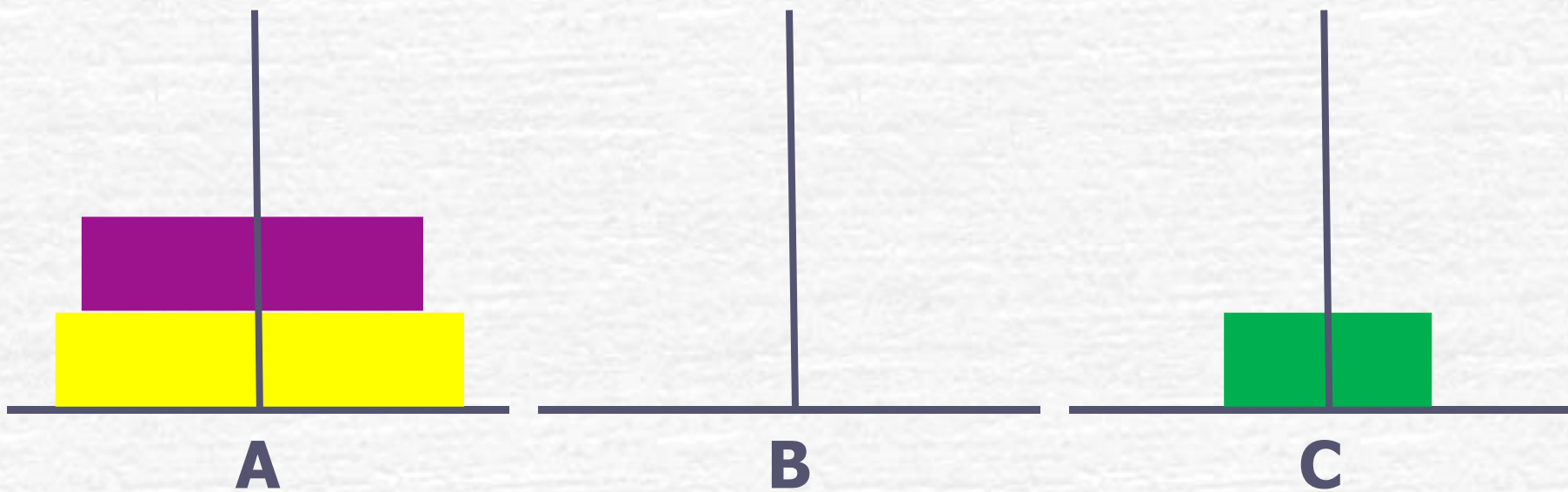
将**2**个盘子从**A**移到**B**的过程

将**1**个盘子从**A**移到**C**



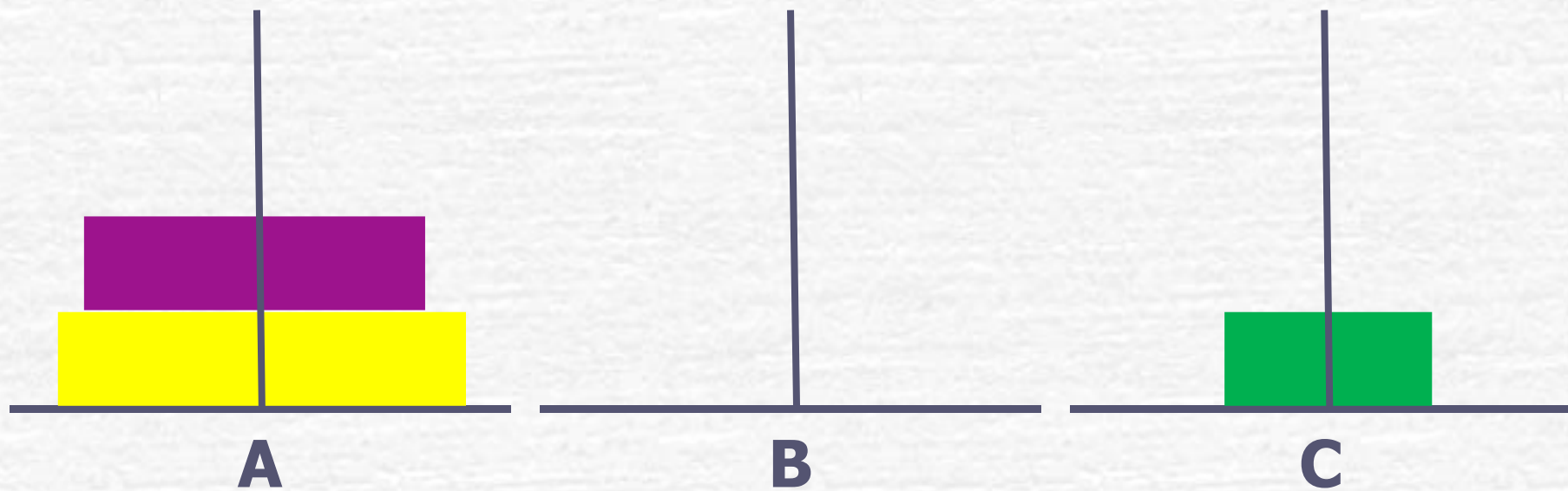
将**2**个盘子从**A**移到**B**的过程

将**1**个盘子从**A**移到**C**



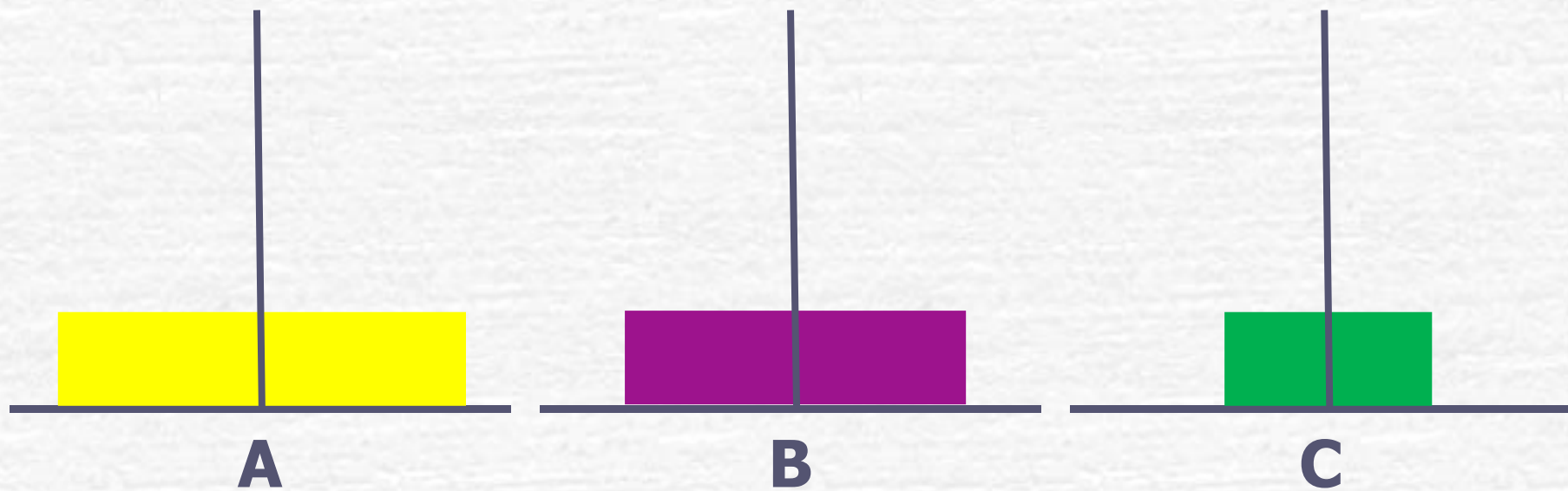
将**2**个盘子从**A**移到**B**的过程

将**1**个盘子从**A**移到**B**



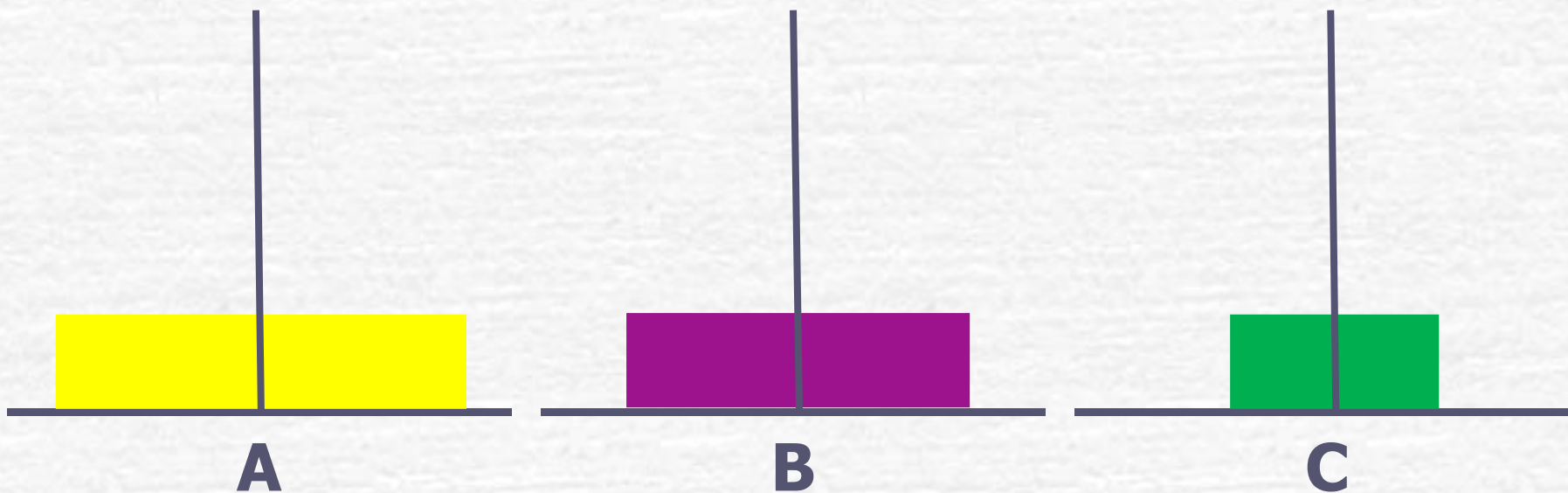
将**2**个盘子从**A**移到**B**的过程

将**1**个盘子从**A**移到**B**



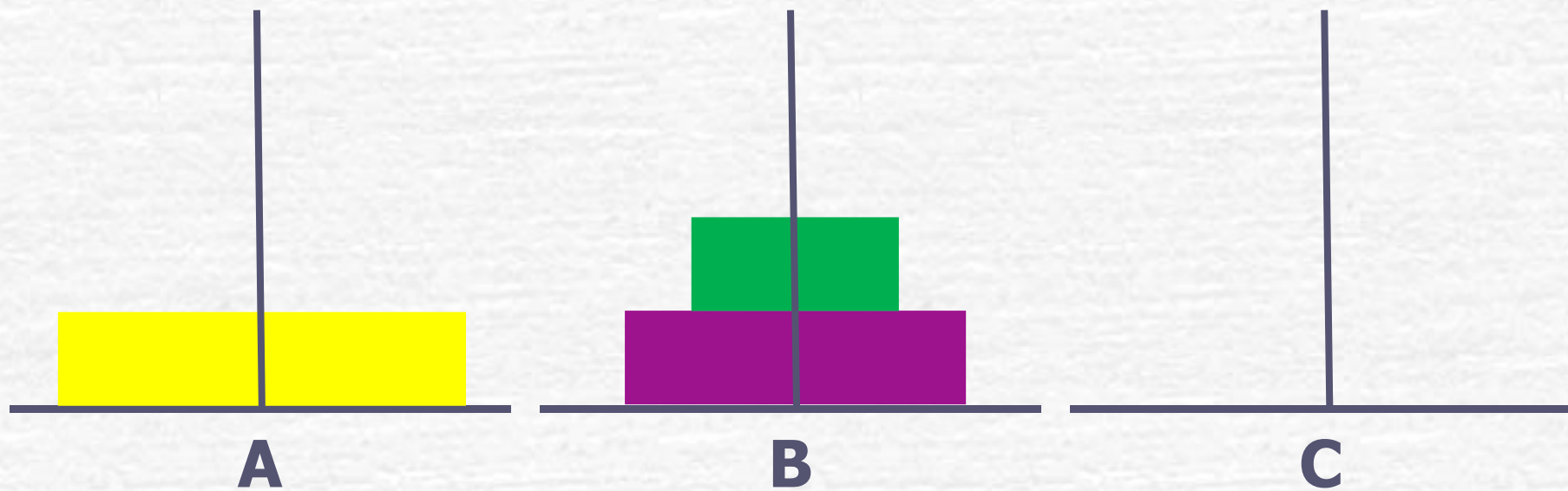
将**2**个盘子从**A**移到**B**的过程

将**1**个盘子从**C**移到**B**

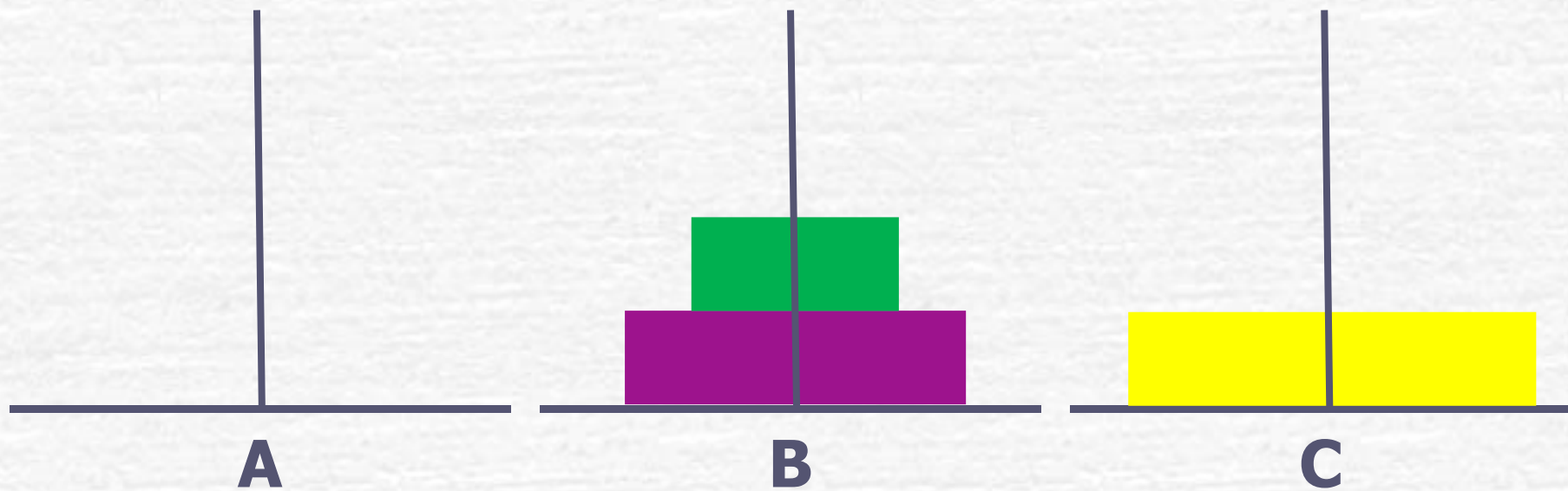


将**2**个盘子从**A**移到**B**的过程

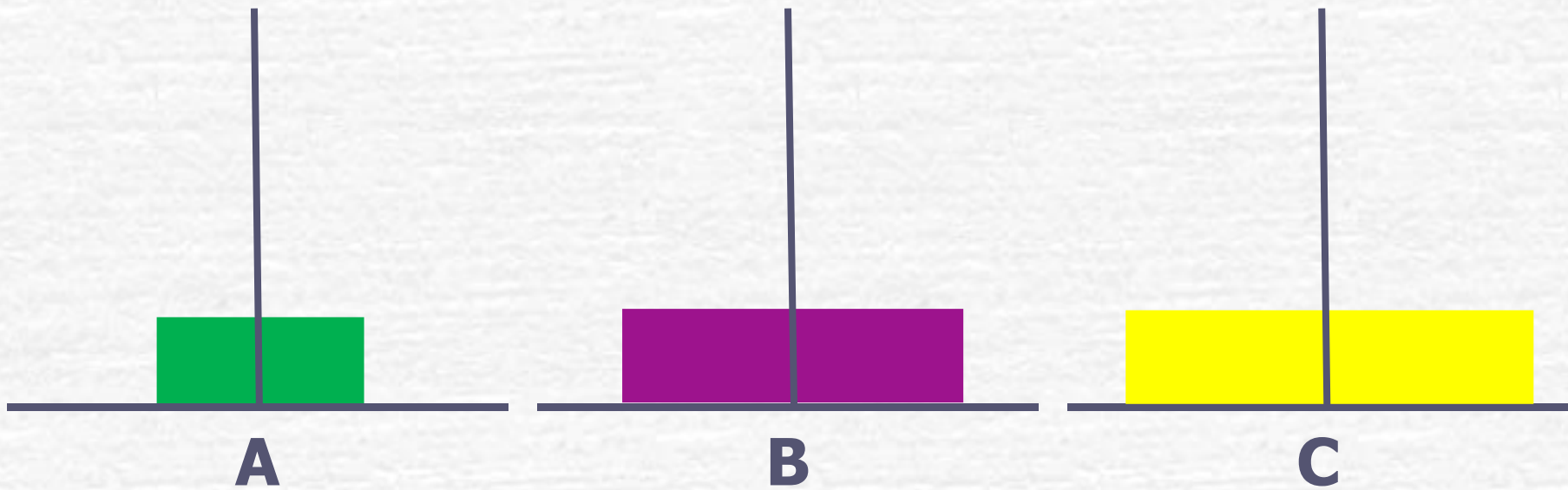
将**1**个盘子从**C**移到**B**



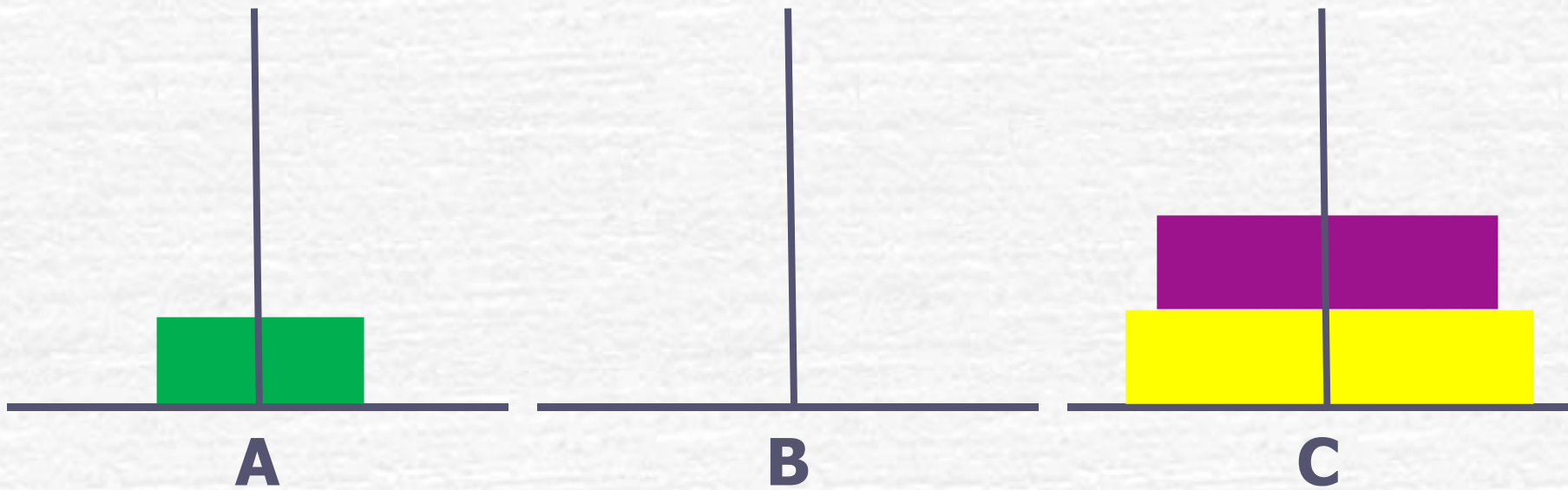
将**2**个盘子从**B**移到**C**的过程



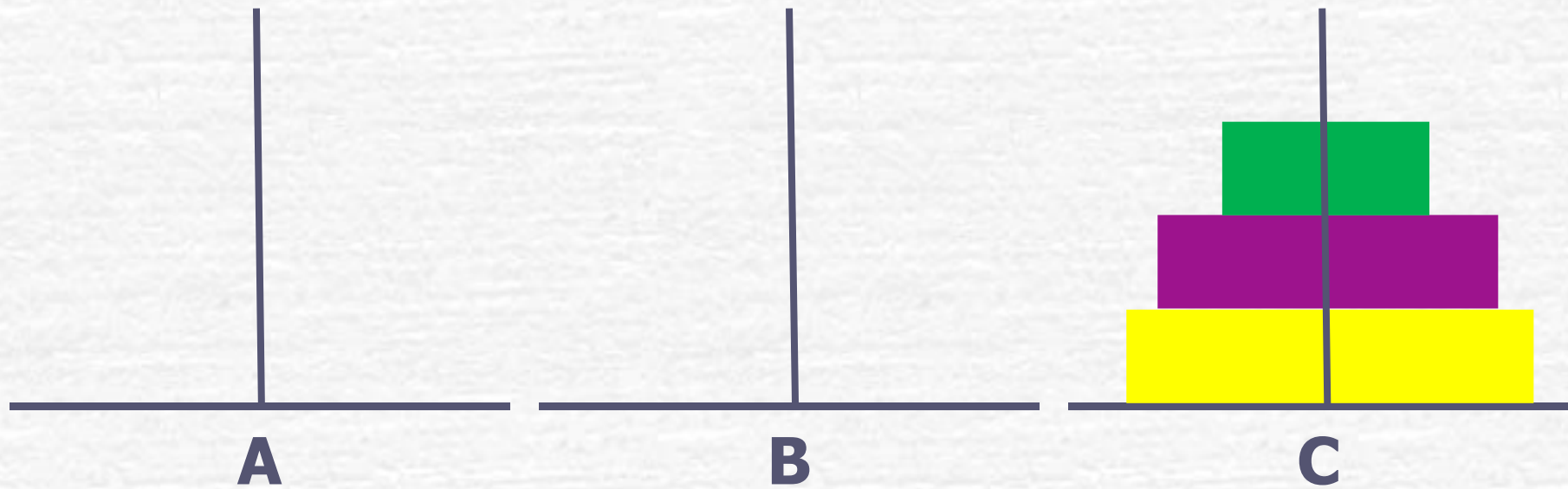
将**2**个盘子从**B**移到**C**的过程



将**2**个盘子从**B**移到**C**的过程



将**2**个盘子从**B**移到**C**的过程



由上面的分析可知：将 n 个盘子从A座移到C座可以分解为以下3个步骤：

- (1) 将A上 $n-1$ 个盘借助C座先移到B座上
- (2) 把A座上剩下的一个盘移到C座上
- (3) 将 $n-1$ 个盘从B座借助于A座移到C座上

可以将第(1)步和第(3)步表示为:

- 将“one”座上 $n-1$ 个盘移到“two”座(借助“three”座)。
- 在第(1)步和第(3)步中, one、two、three和A、B、C的对应关系不同。
- 对第(1)步, 对应关系是one对应A, two对应B, three对应C。
- 对第(3)步, 对应关系是one对应B, two对应C, three对应A。

把上面3个步骤分成两类操作：

- (1) 将 $n-1$ 个盘从一个座移到另一个座上 ($n > 1$)。这就是大和尚让小和尚做的工作，它是一个递归的过程，即和尚将任务层层下放，直到第64个和尚为止。
- (2) 将1个盘子从一个座上移到另一座上。这是大和尚自己做的工作。

编写程序。

- 用**hanoi**函数实现第**1**类操作（即模拟小和尚的任务）
- 用**move**函数实现第**2**类操作（模拟大和尚自己移盘）
- 函数调用**hanoi(n,one,two,three)**表示将**n**个盘子从“**one**”座移到“**three**”座的过程(借助“**two**”座)
- 函数调用**move(x,y)**表示将**1**个盘子从**x**座移到**y**座的过程。**x**和**y**是代表**A**、**B**、**C**座之一，根据每次不同情况分别取**A**、**B**、**C**代入

```
#include <stdio.h>
```

```
int main()
```

```
{ void hanoi(int n, char one, char two, char three);
```

```
    int m;
```

```
    printf("the number of disks:");
```

```
    scanf("%d",&m);
```

```
    printf("move %d disks:\n",m);
```

```
    hanoi(m,'A','B','C');
```

```
}
```

```
void hanoi(int n, char one, char two, char three)
```

```
{ void move(char x,char y);
```

```
    if(n==1)
```

```
        move(one,three);
```

```
    else
```

```
    { hanoi(n-1,one,three,two);
```

```
        move(one,three);
```

```
        hanoi(n-1,two,one,three);
```

```
    }
```

```
}
```

```
void move(char x,char y)
```

```
{
```

```
    printf("%c-->%c\n",x,y);
```

```
}
```

```
the number of disks:3  
move 3 disks:
```

```
A-->C
```

```
A-->B
```

```
C-->B
```

```
A-->C
```

```
B-->A
```

```
B-->C
```

```
A-->C
```

作业：

1) **P218 7.1、7.3**

2) (**www.pintia.cn**) 第7章 函数 (**6-1~6-5**
为基础必做题；**7-1~7-3**为选做题)

【补充】“文件包含”处理

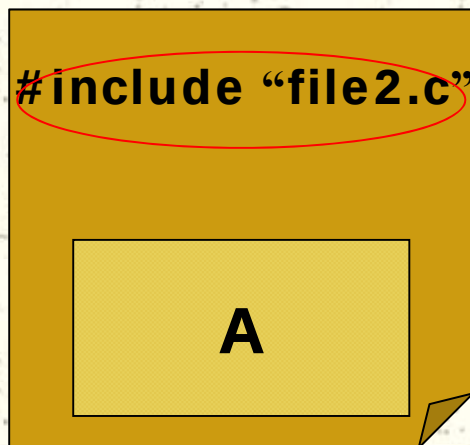
- ✦ 所谓“文件包含”处理是指一个源文件可以将另外一个源文件的全部内容包含进来，即将另外的文件包含到本文件之中。

必须是一个完整的文件名（扩展名不可省略），
可以包含路径

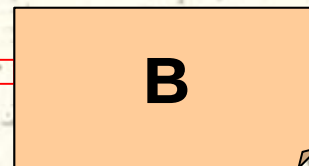
[一般形式]

#include “文件名” 或 **#include <文件名>**

file1.c



file2.c



- 在编译时，文件及被其包含的所有文件将作为一个整体被编译，得到一个目标（.obj）文件。

【补充】“文件包含”处理（2）

- # 文件包含使同一文件可供多个程序共享，可以节省程序员很多重复劳动。
 - 当需要修改这些公用量/函数时，也不必逐一修改各个文件，只需修改这个公用文件即可。
 - 在编译时，文件及被其包含的所有文件将作为一个整体被编译，得到一个目标（.obj）文件。
 - 文件中使用被包含文件中的全局变量、函数时，无需再用extern声明。
 - 被包含文件修改后，凡包含此文件的所有文件都要重新编译。
- # 通常，我们将文件头部的一些信息放在称为“标题文件”或“头文件”的文件中，常以“.h”为后缀。这些信息主要是函数原型、宏定义、全局变量定义和后面要介绍的结构体类型定义。

【补充】“文件包含”处理（3）

【说明】

- # 一个**#include**命令只能指定一个被包含文件，如果要包含 n 个文件，要用 n 个**#include**命令。
- # 两种包含形式**#include** “文件名” 与 **#include** <文件名> 的差别在于搜索路径：
 - 尖括号指示预处理程序到C库函数头文件所在的目录中搜索；（标准方式）
 - 双引号则指示预处理程序先为用户当前目录中查找，找不到时才按标准方式查找。

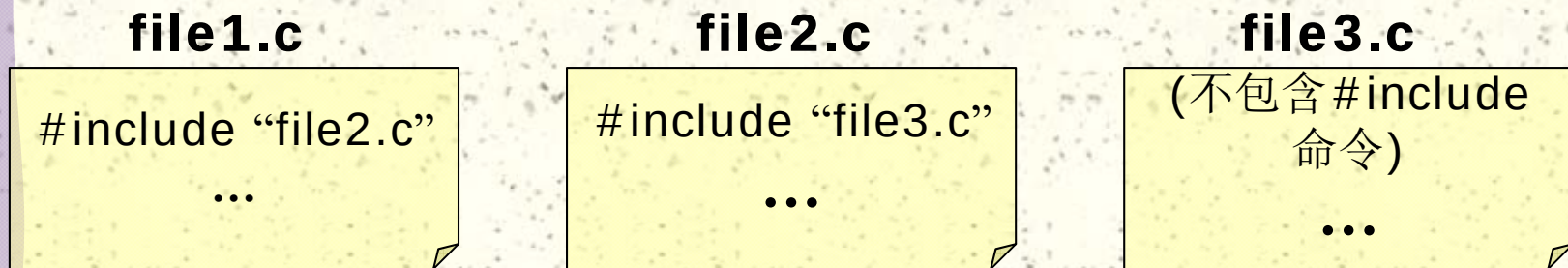
【补充】 文件包含（4）

【说明】

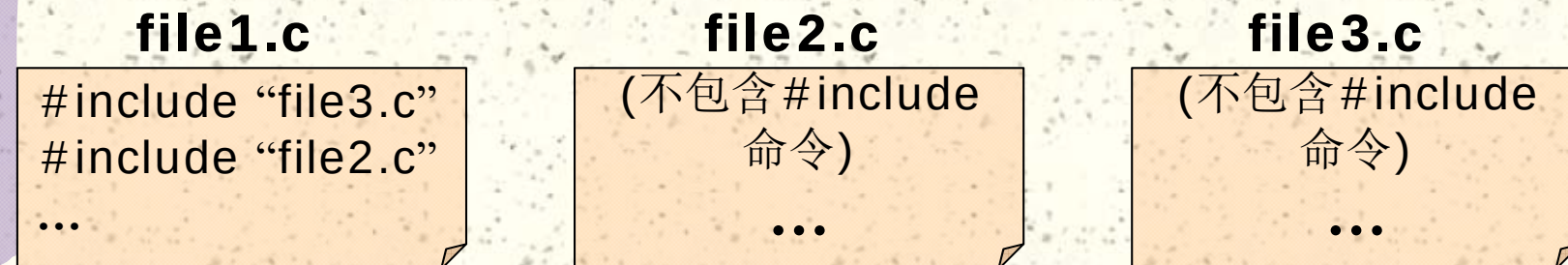
在一个被包含文件中又可以包含另一个被包含文件，即文件包含是可以嵌套的（ ≤ 8 层）。

【例】 文件1包含文件2，而文件2中要用到文件3的内容：

方法一：



方法二：



作业：

1) P218 7.4, 7.7

2) 补充题（见下页）

2) 从键盘输入一个整数 m ($4 \leq m \leq 20$), 输出如下的 m 阶整数方阵 (保存在二维数组中)。
 例如, 若输入 “4” 和 “5”, 则分别输出 (输出在main中实现)

16	9	4	1	25	16	9	4	1
9	4	1	16	16	9	4	1	25
4	1	16	9	9	4	1	25	16
1	16	9	4	4	1	25	16	9
				1	25	16	9	4

请在下面框架的基础上写出完整的程序。

```
#define M 20
```

```
void Matrix(int n, int xx[][M]) { }
```

```
main()
```

```
{
```

```
    Matrix(m, aa);
```

```
}
```